Native Rectangular Linear Assignment with Implicit Rejection: A Jonker–Volgenant Solver without Square Expansion

Ratha Siv[1], Raphael Reme[2], Matei Mancas[1], Bernard Gosselin[1]
[1]University of Mons, Belgium
[2]Institut Pasteur, France

31 October 2026

Abstract

Rectangular assignment with a uniform unmatched penalty is often implemented by expanding the cost matrix. We integrate established rectangular Jonker–Volgenant principles and implicit rejection with float64 working arithmetic and reusable workspaces in Rust (JVRUST), hybrid Rust/C++ (JVHYBRID) and C++ (JVCPP). A campaign evaluates laprust/lapcpp 1.2.0 and LAPX 0.13.0 in four Windows/Linux and Python environments. Each environment contains 302 cases: 3,286 complete-call timing conditions with five inputs and three rounds, plus 91 memory conditions with five fresh-process trials. Against the matching compact SciPy reference, all three native implementations exceed a 3% advantage on all 102 required cases; JVRUST and JVCPP do so on 163–165 and 163–166 of 166 optional cases, respectively. Controlled comparisons qualify these gains by representation, initialization, sparsity, batching and orientation. Assignment-replay gains attenuate in tracker updates, and equal replay objectives do not imply identical tracking outputs. A separate 67-case Windows LSAPE study examines related transformations within Hungarian code. The contribution is software integration and reproducible performance characterization. Adverse results are retained; no algorithmic priority, statistical significance or universal solver ranking is claimed. The initialization policy was developed using related workload families, so the campaign is not a held-out test of heuristic optimality.

Keywords: linear assignment; Jonker–Volgenant; rectangular matching; implicit rejection; rejection penalties; research software; Rust; C++; Python

1 Introduction

Multi-object trackers associate tracks with detections whose number can differ, while allowing an object on either side to remain unmatched. ByteTrack [1] and related association code express this through the `cost_limit` argument of `lap.lapjv` [2]. Its padded implementation embeds the $N \times M$ costs in an $(N + M)$-square matrix. The limit prices rejection; it does not require maximizing the number of below-threshold pairs.

For float64 costs with 256 rows and 4096 columns, the original matrix occupies 8 MiB, square zero-padding requires 128 MiB, and optional-assignment expansion requires 144.5 MiB. These are cost-buffer dimensions, before workspace and output. Compact representations avoid this expansion, but their performance still depends on initialization, cost structure, orientation and interface work.

Rectangular JV, outsourcing and assignment with insertion and deletion costs already provide compact formulations and specialized searches [3–10]. Our contribution is to integrate these principles into Python-facing Rust, hybrid Rust/C++ and C++ implementations, then evaluate their complete calls against equivalent dense and sparse formulations. SciPy's rectangular solver already avoids required-assignment padding, and exact capped costs provide an original-dimension optional baseline. Neither dimensional savings nor implementation language alone explains the measured differences.

The paper contributes a self-contained derivation and optimality certificate for uniform-penalty assignment; implementations combining bounded reductions, a reusable rejection terminal and batch workspaces; and an empirical study with explicit representation, policy, work, orientation and application controls. Four environments retain separate measurements and a practical ±3% tie band. The study includes unfavorable cases, planned coverage limits and the distinction between assignment latency and tracker-update latency. A separate LSAPE experiment examines related transformations within Hungarian code, without comparing JV and LSAPE runtimes.

Sections 2–4 give the background, formulation and implementation. Section 5 specifies the saved campaign, Section 6 reports its results, and the discussion and limitations delimit their interpretation. Appendices organize additional evidence by workload and measurement question. The indexed supplement retains absolute values and provenance for every plotted observation.

2 Background and related work

2.1 Assignment algorithms and implementation lineage

Jonker and Volgenant's algorithm combines initialization, reductions, and a specialized shortest augmenting path procedure [3]. Initial stages often construct much of the matching cheaply. Remaining free rows require augmenting paths that can reassign existing pairs. A square matrix therefore does not eliminate the augmentation phase; it only removes the need for a dimensional transformation.

We use completion to mean the augmenting-path phase that finishes a partial matching. It does not refer to padding the cost matrix.

The baselines are established Jonker–Volgenant families, evaluated through their LAPX implementations [11]. JVX (`lapjv`, `lapjvx`, `lapjvxa`) uses the gatagat/lap-derived dense core [2]; the dense campaign times `lapjvxa`, which returns assignment pairs. JVS (`lapjvs`, `lapjvsa`) uses the src-d/lapjv-derived implementation [12]. These implementations have different initialization and numeric details. Their shared use of augmenting paths does not imply identical kernels.

SciPy [13] supplies a native rectangular solver documented as a modified Jonker–Volgenant algorithm without initialization, referencing Crouse's treatment [7, 14]. Initialized rectangular JV also predates this work: Volgenant's RJV omits square column reduction and applies augmenting row reduction from zero column potentials [4]. Bijsterbosch and Volgenant refine RJV and specialize it for k-cardinality and outsourcing, including a capped-cost comparison on the original dimensions and a single relevant external option per person [5]. That unexpanded outsourcing formulation originates with Vander Wiel and Sahinidis, whose shortest augmenting path algorithm treats external assignment through a single unrestricted arc [6]. These initialized rectangular and outsourcing searches are direct predecessors of the state and rejection handling studied here.

2.2 Optional assignment and compact representations

Optional assignment also has clear precedents. Bougleux and Brun formulate linear sum assignment with edition using element-specific insertion and removal costs and a null element on each side [8]. Their adapted Hungarian algorithm permits augmenting paths to terminate at a null element, representing removal or insertion. The associated LSAPE toolbox supplies C++ implementations, including null-terminal path routines [15]. The 2016 publication is the conceptual comparison; the July-2017 LSAPE-v4 release is the implementation used in our separate controlled experiment. Equality with the author-hosted v4 archive was checked on 27 October 2026, but does not establish identity with the code behind the 2016 timings.

Jones, Chawdhary, and King adapt Volgenant–Jonker methods to graph-edit matching [9]. Their compact representation avoids explicitly storing the redundant zero and infinite regions of the expanded matrix, using only (n+1)(m+1)-1 entries. Bougleux and colleagues subsequently describe a compact reduction from

assignment with error correction to a rectangular assignment problem without cost constraints of the earlier reductions [10].

The present solver specializes the penalty structure: a common cost $L/2$ per unmatched endpoint yields a one-sided objective with rejection cost L per unmatched smaller-side row, up to an additive constant (Section 3.3). Within JV code descended from the JVS lineage, we combine rectangular reductions, rejection state stored as one distance and one predecessor row, the threshold arguments of `lap.lapjv`, and native batching. These established ingredients define a software-integration contribution; we claim no new optional-assignment objective, null-terminal principle, algorithmic priority or asymptotic improvement over the compact predecessors. Table 2 defines the measured interfaces and LSAPE arms. The separate experiment in Section 6.9 evaluates one-sided adaptation within LSAPE-v4 and makes no JV-versus-LSAPE runtime comparison.

Application interfaces also accept non-assignment costs. MATLAB's `assignjv`, `assignmunkres`, and `assignauction` take a scalar cost of non-assignment charged per unmatched endpoint [16]. The Tracker Component Library includes a Jonker–Volgenant routine for missed detections that never builds the augmented matrix but keeps search state for one virtual row per target [17].

The compact reduction in [10] is also an executable comparison point. For uniform endpoint penalties, its cost factorization reduces, up to a constant shift, to capping real costs at the combined rejection price. Section 3.5 gives a self-contained argument for this specialization, including finite signed costs, and Section 6.3 evaluates it with SciPy. This provides a baseline without square expansion; it does not reproduce every optimization in the authors' LSAPE software.

2.3 Interfaces and transformed problems

JVRUST, JVHYBRID and JVCPP provide native required rectangular assignment and finite-limit rejection. JVX uses square padding or augmentation, and JVS receives a square problem through an adapter. SciPy receives the required, capped, dummy-column, diagonal-dummy or square-expanded representation named in each comparison. The required reference is ordinary rectangular SciPy; the optional reference is capped SciPy throughout the campaign.

The optional sparse comparisons include SciPy's `min_weight_full_bipartite_matching` (CSGRAPH) [18] and LAPX's `lapmod` (LAPMOD). Their measured adapters construct admissible sparse edges and rejection structure from the same dense input. The layout adaptations credit Pylapy [19]; the benchmark source retains its MIT notice and identifies the adapted revision. Thus their complete-call times include representation construction. Table 2 distinguishes these numerical contracts from the separate LSAPE arms.

3 Problem formulation

3.1 Required rectangular assignment

Let C be an N × M real cost matrix, with forbidden edges excluded from the admissible edge set E. Orient the matrix so that r = min(N,M) and c = max(N,M), with r ≤ c. In (1) and (4), $x_{ij} = 0$ for $(i, j) \notin E$. A required rectangular assignment solves

$$\min_{x} \sum_{(i,j)\in E} C_{ij}x_{ij}, \quad \sum_{j} x_{ij} = 1, \quad \sum_{i} x_{ij} \le 1, \quad x_{ij} \in \{0, 1\}. \tag{1}$$

The solver must match all r rows if this is feasible. Surplus columns remain unused. If the original matrix is tall, the returned indices are mapped back to the original orientation.

3.2 Finite-limit assignment

Let L be a finite limit and A a matching with K pairs. The finite-limit objective is

$$F(A) = \sum_{(i,j)\in A} C_{ij} + \frac{L}{2}(r + c - 2K). \tag{2}$$

Each unmatched vertex incurs L/2. In the square augmentation used by JVX, the real costs occupy the upper-left block. Real-to-dummy and dummy-to-real blocks cost L/2, while the dummy-to-dummy block is zero.

Equation (2) is not equivalent to solving a required assignment and then discarding expensive pairs. For example, let

$$C = \begin{pmatrix} 0 & 0.9 \\ 0.9 & +\infty \end{pmatrix}, \qquad L = 1.$$

Two real pairs cost 1.8. Selecting only the zero-cost pair leaves two unmatched vertices and costs 1.0 under (2). Both nonzero finite edges are below the threshold, yet the smaller matching is preferable.

At equality, a real edge of cost L can tie with rejecting both endpoints. The objective does not impose maximum cardinality among equal-cost optima. Applications that require a particular inclusive-threshold or cardinality rule must specify it separately.

3.3 One-sided rejection equivalence

Proposition 1. In exact arithmetic, minimizing (2) is equivalent to minimizing

$$P(A) = \sum_{(i,j)\in A} C_{ij} + L(r - K) \tag{3}$$

on the oriented real rectangular problem.

Proof. Rearranging the rejection term gives

$$F(A) = \sum_{(i,j)\in A} C_{ij} + L(r - K) + \frac{L}{2}(c - r).$$

The final term is independent of A. Consequently, F and P have the same minimizers. The argument holds for any finite real L, including zero and negative values, and includes empty dimensions. ∎

The transformation is useful because every smaller-side row can either select a real column or take a rejection option costing L. No separate dummy column is needed for each possible rejection. The constant must be restored when comparing the full penalized objective, but it does not affect the optimizer.

Signed support is an interface convenience rather than a new uniform-penalty model: adding a constant k to all admissible real costs and to L shifts P by the constant kr in exact arithmetic. Floating-point execution still requires the numeric contract below.

The public returned total is different again: it sums only selected original input costs and excludes rejection penalties. Our validation and benchmark checks reconstruct the penalized objective rather than confusing it with this reported total.

3.4 Primal-dual certificate

For an effective row penalty p, introduce a rejection variable z_i:

$$\begin{aligned} \min_{x,z} \quad & \sum_{(i,j)\in E} C_{ij}x_{ij} + p\sum_i z_i \\ \text{subject to} \quad & \sum_j x_{ij} + z_i = 1, \qquad \sum_i x_{ij} \le 1, \qquad x_{ij}, z_i \ge 0. \end{aligned} \tag{4}$$

This is a network-flow formulation: each row contributes one unit, real columns have unit capacity, and a row may send its unit directly to the terminal at cost p. The integral capacities yield integral optimal solutions. Its dual is

$$\max_{u,v} \sum_i u_i + \sum_j v_j, \quad u_i + v_j \le C_{ij} \quad ((i,j) \in E), \quad u_i \le p, \quad v_j \le 0. \tag{5}$$

Proposition 2. A feasible matching/rejection solution is optimal if a feasible dual satisfies tightness on matched real edges, $u_i = p$ for rejected rows, and $v_j = 0$ for unused columns.

Proof. The primal-dual gap can be written as

$$P - D = \sum_{(i,j)\in E} (C_{ij} - u_i - v_j)x_{ij} + \sum_i (p - u_i)z_i + \sum_j v_j \left(\sum_i x_{ij} - 1 \right). \tag{6}$$

Every term is nonnegative by primal and dual feasibility. The stated conditions make each term zero, so the primal and dual objectives coincide. Weak duality establishes optimality. ∎

For required rectangular assignment, rejection variables and the constraints $u_i \le p$ are omitted. The non-positive column potentials and zero potentials on unused columns remain important. Ordinary square reductions cannot be transferred blindly to a rectangular implementation.

These propositions establish the formulation and a final optimality certificate. They are not a formal verification of every implementation transition or a floating-point error bound.

3.5 Compact equivalent formulations

The one-sided objective also permits two formulations for an ordinary required rectangular solver. First, append r dummy columns of cost L to the oriented real matrix. The resulting r × (c+r) problem assigns every row either to a real column or to one of the distinct dummy columns. Removing dummy pairs recovers an optimal finite-limit matching under (3).

Second, no additional columns are necessary when rejection costs are uniform. Define a capped matrix on the original r × c dimensions:

$$\widehat{C}_{ij} = \min(C_{ij}, L), \qquad \min(+\infty, L) = L. \tag{7}$$

Proposition 3. Let B be a minimum-cost required matching of all r rows under (7). Retain only its pairs whose original cost is strictly below L. The retained matching minimizes (3); rejecting equality ties is one valid optimality convention.

Proof. For any required matching B, let A be its retained subset. Every discarded pair costs L in the capped problem, so the capped cost of B is exactly P(A). Conversely, any finite-limit matching A can be extended to r pairs because c≥r. Each of the added pairs costs at most L after capping, and each original pair costs at most its original cost. Thus some required matching has capped cost at most P(A). Taking minima in both directions proves equality of the optimal values and optimality of the retained subset. The argument holds for finite real L and finite admissible costs, including negative values; forbidden original pairs are discarded. ∎

This is the uniform-penalty case of the substitution factorization underlying [10], stated here to make the comparison reproducible. Under this specialization, Bougleux et al. (Theorem 1, Eqs. (7)–(9)) retain selected substitutions at equality, whereas Proposition 3 and the capped SciPy adapter discard them. Both conventions preserve the optimal penalized objective in exact arithmetic, but may return different pairs and cardinalities. Capping occurs before optimization. It is therefore different from solving the original required problem and then discarding expensive pairs, which fails in the example in Section 3.2. The capped and dummy-column baselines are equivalent in exact arithmetic; their floating-point behavior is assessed separately by the validation experiments.

Table 2 gives the representation dimensions and objectives. A reusable terminal is a search option, not a capacity-one dummy column. These dimensions describe buffers, not measured process memory.

4 Implementation

4.1 Shared architecture and three implementations

Each implementation consists of a thin Python extension and a native core. The extension validates shape, real numeric type, precision, and limit options, prepares a contiguous working representation where necessary, and invokes the core. The single-matrix and batch interfaces, return contracts, and error behavior are identical across the three implementations. JVRUST is provided by laprust 1.2.0 and the `lapjvr` family in LAPX 0.13.0; JVHYBRID by LAPX `lapjvh`; and JVCPP by lapcpp 1.2.0 [11, 20, 21].

Interface compatibility here means that the single-matrix entry points accept the `extend_cost`, `cost_limit`, and `return_cost` arguments of `lap.lapjv` with the same objective: the finite limit prices unmatched endpoints, and the reported total excludes those penalties. They return matched row and column index arrays, or pairs, rather than lap's full-length per-row and per-column assignment vectors, so callers of lap need a small output adaptation. Compatibility does not imply identical assignments when optima tie, or identical behavior across precision modes. At $C_{ij} = L$, matching and rejection tie in exact arithmetic. Native reductions and search prefer rejection on computed ties, whereas the evaluated padded `lap.lapjv` can retain the edge. This is not a strict $C_{ij} < L$ output guarantee: the rounded effective penalty p in Section 4.5 can differ from the supplied L.

The native dispatcher selects the square kernel only for square inputs with an infinite limit. The square kernel's initialization and reductions are followed by an augmentation phase that completes the matching. JVRUST executes this phase in Rust. JVHYBRID calls a C++ completion kernel that operates on Rust-owned workspace and preserves the completion strategy and operation order. JVCPP implements the whole algorithm in C++, including the same C++ completion routine. This design permits two comparisons: of completion kernels within otherwise identical Rust code (JVRUST against JVHYBRID), and of complete Rust and C++ implementations of one algorithm (JVRUST against JVCPP).

All rectangular inputs and finite-limit inputs, including square matrices with a finite limit, use the rectangular core. JVRUST and JVHYBRID share that Rust core, so differences between them on these paths cannot be interpreted as evidence of different rectangular algorithms. JVCPP executes the same rectangular algorithm in C++.

4.2 Native rectangular reductions and search

The rectangular core stores separate mappings, potentials, predecessors, distances, and free-row lists. Following Volgenant's RJV initialization [4] and its later refinements [5], it omits square column reduction and starts column potentials at zero. Its bounded augmenting row reductions lower a column potential only while assigning that column, maintaining non-positive potentials and zero potentials on unused columns.

After reductions, each remaining free root is introduced with a feasible row potential obtained from its available reduced costs and, in finite-limit mode, the rejection penalty. The root's reduced costs are retained as the initial shortest-path distances, so the first relaxation does not read the cost row again. A shortest-path search then scans real columns. Reaching a free column completes an augmenting path. Reaching an occupied column continues through its assigned row, allowing previous pairs to be rearranged.

The implementations use two bounded rectangular row-reduction passes with $B = \min(c, 4)$. Each pass permits at most B times its initial free-row count in row scans, including displacement retries. Among equally cheap real columns, a free column is preferred [5]; in finite-limit mode the implicit rejection option still wins its computed ties. For required rectangular matching, $\max(r, 8)$ consecutive displacements without acquiring a free column disable further price lowering. Acquiring a free column resets the streak until the cutoff is reached; once disabled, the cutoff persists across both passes. The current scan completes normally.

Finite-limit inputs, including optional squares, use the same scan allowance and tie preference without that stall cutoff. Remaining free rows enter augmenting search. These rules limit initialization work without removing admissible edges or changing the objective. Required square problems retain their separate square initializer, which samples four complete rows to select an existing initialization strategy.

Policy selection used development workload families, including congested and tied costs, and is not a held-out validation of an optimal heuristic. The required-only cutoff limits unproductive displacement chains while allowing optional chains to continue within the scan allowance. Free-column tie scans can add work on heavily tied costs. Separately built policy variants test these choices within matched implementations (Section 6.4); they do not assign a causal share of the native-versus-SciPy difference to any rule.

4.3 Implicit rejection terminal

For finite-limit matching, the search compares real-column paths with a rejection terminal. A real edge has reduced cost $C_{ij} - u_i - v_j$. Leaving a row unmatched has reduced cost $p - u_i$. The implementation stores the best terminal distance and its predecessor row using the variables `dummy_distance` and `dummy_row`.

More precisely, let δ_i be the shortest-path distance at which row i is reached; the root has $\delta = 0$. When row i is reached, the rejection candidate $\delta_i + p - u_i$ updates the terminal distance $D_\perp$, the smallest such value so far, and its row. Each unsettled real column j is relaxed to $\min(d_j,\ \delta_i + C_{ij} - u_i - v_j)$. If $D_\perp$ is no greater than the smallest unsettled column distance, rejection is selected; otherwise that column is settled, and the search either ends at a free column or continues through the column's matched row. The terminal therefore adds one distance and one predecessor row to the ordinary search state.

Let D be the distance of the selected endpoint. The root's potential increases by D. Every settled column j has its potential lowered by $D - d_j$ and, if matched, its row's potential raised by the same amount. Assignments are then reversed from the free column, or from the terminal's predecessor row, back to the root. A free column ending the path has $d_j = D$ and keeps its zero potential, so column potentials stay non-positive and unused columns stay at zero.

If rejection is selected, the number of real pairs need not increase: the path can replace one row's real assignment and leave another row unmatched.

Figure 1 illustrates the two endpoints, and Algorithm 1 places them within the complete rectangular solve.

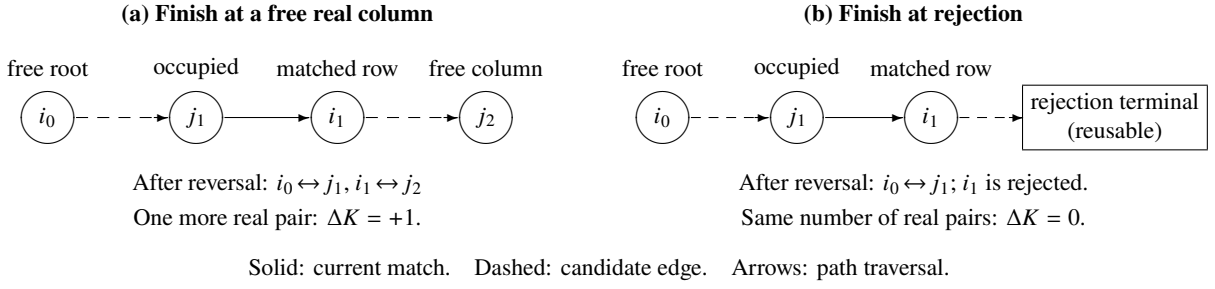


Figure 1: Conceptual paths to a free real column or reusable rejection terminal. A free-column endpoint adds a real pair; a rejection endpoint can transfer unmatched status without increasing cardinality. This illustrates the algorithm rather than an observed execution trace.

```
Orient C so that r <= c.
Initialize mappings, potentials, and free-row state.
Apply bounded row reductions, including rejection choices in finite-limit mode.
For every row still requiring processing:
    Establish the root row potential from real reduced costs and finite p.
    Search shortest alternating paths over real columns.
    In finite-limit mode, also track the best rejection terminal.
    Stop at a free real column or the selected rejection terminal.
    Update potentials and reverse assignments along the path.
Map real pairs back to the original matrix orientation.
```

Algorithm 1: High-level native rectangular control flow with implicit rejection. The implementation additionally specifies reduction budgets, finite-value checks, tie handling, and workspace reuse.

4.4 Storage, batching, and operation bounds

For contiguous wide inputs in the selected cost dtype, the real cost matrix can be borrowed. Solver state is $O(r + c)$, with float64 numerical workspace even when costs are float32; widening this linear workspace does not require a dense float64 cost copy. Pair output is $O(r)$. Tall inputs may require an $O(rc)$ orientation copy, and dtype or layout conversion can also require a dense copy. Including these costs, storage remains $O(rc + r + c)$, rather than requiring a c-square or (r+c)-square cost matrix.

Batch input storage is shared with the call. Each active worker retains its own workspace and can reuse it across problems; results are collected in input order, and the lowest failing index is reported on error. JVRUST and JVHYBRID schedule problems on reusable Rayon thread pools. JVCPP uses a reusable C++ thread pool and claims consecutive small problems in chunks, bounded to about 4096 costs per claim so that adjacent difficult problems are not serialized on one worker. Wide inputs require no per-worker padded cost matrix; tall problems can require orientation buffers. This distinction is relevant to peak memory as worker counts increase.

Each bounded row-reduction pass performs $O(rB)$ scans of at most c real costs; the additional free-column tie scan has the same order. Since $B \leq 4$, the two passes cost $O(rc)$. Each final augmenting search visits at most r occupied real columns before reaching a terminal, scanning up to c candidates at each step. Across r roots, augmentation costs $O(r^2c)$. Thus $O(rc + r^2c)$ is a conservative bound for this release's rectangular solve, excluding separately described conversion and output work. The required-only cutoff can reduce initialization work but is unnecessary for this bound. This is an implementation bound, not an asymptotic improvement over established rectangular assignment methods.

4.5 Numeric and interface contract

All three native implementations retain selected float32/float64 cost storage while using float64 potentials, distances, minima, and reduced-cost arithmetic on required square, rectangular, and finite-limit paths. LAPX's JVS implementation also widens its numerical working state. The wider working arithmetic reduces cancellation when large costs coexist with small assignment differences; it cannot restore distinctions lost during input conversion or guarantee exact optimization for arbitrary dynamic ranges. Finite float64-to-float32 conversion overflow is checked. Other accepted real numeric dtypes first undergo NumPy conversion to float64 without an equivalent explicit overflow check; on platforms with wider real types, a finite value outside binary64 range can become positive infinity at that stage. A motivating upstream report [22] gives the exactly representable float32 matrix $\begin{pmatrix} 2^{24} & 2^{24} \\ 1/8 & 1/4 \end{pmatrix}$: choosing the diagonal costs $2^{24} + 1/4$, while the off-diagonal costs $2^{24} + 1/8$. Accumulating selected original costs in float64 exposes the difference that float32 arithmetic can conceal. This witness motivates regression coverage without establishing robustness for every numerical scale.

For a finite limit, the boundary rounds the half-limit in the selected cost dtype T before promoting it for working arithmetic: $h = \text{round}_T(L/2)$, $p = \text{float64}(h) + \text{float64}(h)$. The effective row penalty p need not

equal the supplied L. The theoretical equivalence therefore applies to the represented costs and effective penalty. The controlled performance study uses float64 inputs; the application study retains float32 tracker costs and separately widens the same values for replay (Appendix D). Saved validation checks the represented objective and targeted numerical and interface cases; it does not establish arbitrary-range exactness. Native output does not apply a raw $C_{ij} < L$ post-filter, so computed-tie preference does not guarantee rejection whenever the raw cost equals the supplied limit. Public totals sum selected original input costs and exclude rejection penalties.

Positive infinity represents a forbidden edge. NaN and negative infinity in the matrix are outside the documented supported contract. Required matching can be infeasible, and reduced-cost overflow can also cause an error; these conditions should not be conflated. As defensive checks, a path reversal that does not reach its root and a final matching whose row and column mappings disagree are reported as internal errors rather than returned.

All methods in the controlled assignment timing suites return int64 pair arrays and omit total-cost calculation. Pair ordering remains native to each interface. Untimed validation checks feasibility and objective values, rather than demanding identical arrays when optimal assignments tie.

5 Experimental design

5.1 Environment, placement and measurement boundary

The light campaign contains one session in each environment: Linux/Python 3.11.18 (L311), Linux/Python 3.14.10 (L314), Windows/Python 3.11.18 (W311), and Windows/Python 3.14.10 (W314). This order is used throughout. All sessions measure laprust 1.2.0, lapcpp 1.2.0, LAPX 0.13.0, NumPy 2.4.6 and SciPy 1.17.1. The shared dependency versions avoid confounding the interpreter comparison with different NumPy/SciPy releases. As of 31 October 2026, SciPy 1.17.1 is the newest non-yanked stable release whose declared Python requirement admits both interpreters; 1.18.1 requires Python ≥ 3.12 [23]. The rectangular assignment source/header and sparse matching source are byte-identical between those two tags [24]; this source check does not establish identical builds or timings. Version numbers and native-module hashes are recorded per environment; the comparisons do not isolate interpreter or operating-system causality.

Both operating-system records identify an Intel Core i9-13900KS. The launcher verifies that logical CPUs 0, 2, 4 and 6 belong to four distinct performance cores; single-threaded work uses CPU 0. Windows records the Balanced power scheme and Linux the powersave governor. Clock frequency and background activity are observed rather than fixed. The saved topology establishes physical-core separation for the selected logical CPUs.

One command runs assignment, diagnostics and application stages, with Windows additionally running the frozen LSAPE study. Recorded total launcher times are 66.23, 65.39, 85.71 and 85.32 minutes in L311/L314/W311/W314, including the recorded preparation state and suite stages. These are completed elapsed times on the recorded configurations, including their existing preparation state; they do not predict a clean-machine build time. Primary timings cover complete calls: input conversion and representation construction where required, allocation/workspace handling, solving and pair output. Input generation and untimed reference validation are outside the timing boundary. Constructor-only and instrumented measurements are secondary evidence and are never substituted for complete-call latency.

5.2 Workload coverage and estimators

Each environment has 302 cases: 268 single matrices (102 required and 166 optional), 14 batch/workspace workloads and 20 memory cases. The 3,377 planned jobs comprise 3,286 timing conditions and 91 memory conditions. All are complete, with no malformed records or unobserved planned timing conditions. Geometry contributes 120 single-matrix cases; the remaining families cover base inputs, formulations, size and

Table 1: Protocol and evidence inventory. Every column is a separate environment; repeated rounds are within-session observations. Timing and memory conditions have different measurement contracts.

Attribute	L311	L314	W311	W314
Python	3.11.18	3.14.10	3.11.18	3.14.10
Operating system	Linux	Linux	Windows	Windows
Single/batch logical CPUs	0 / 0, 2, 4, 6; distinct physical performance cores			
laprust / lapcpp / LAPX	1.2.0 / 1.2.0 / 0.13.0			
NumPy / SciPy	2.4.6 / 1.17.1			
Inputs / rounds / target	5 / 3 / 20 ms per dense timing block			
Single required / optional	102 / 166 cases per environment			
Batch/workspace / memory	14 / 20 cases per environment			
Completed/planned timing	3,286/3,286	3,286/3,286	3,286/3,286	3,286/3,286
Memory conditions / trials	91 / 5 fresh processes per condition			
Application pipelines / rounds	8 / 3; 999 frames per pipeline			
Captured matrices / precisions	7,539 / float32 and widened float64			
Replay block target / warmup	30 ms / 32 tracker frames			
LSAPE cases / processes	—	—	67 / 3	67 / 3
LSAPE inputs / rounds / target	—	—	4 / 9 / 30 ms	4 / 9 / 30 ms

aspect-ratio scaling, large and extreme shapes, congestion, forbidden entries, ties, signed costs and rejection limits.

Non-geometric synthetic costs start from independent $U_{ij} \sim \text{Uniform}[0, 1)$. Signed costs use $2U - 1$. The congested-block family adds 3 to the two off-diagonal blocks formed by the first $3r/4$ rows and $c/4$ columns (integer floors); the congestion sweep varies the row fraction over $0.25, 0.5, 0.625, 0.75, 0.9$ at column fraction 0.25. Quantized costs use $\lfloor kU \rfloor / k$ for $k \in \{1, 2, 8, 256\}$. The forbidden-entry sweep masks independent entries with probabilities $0, 0.5, 0.9, 0.99$ while retaining a feasible diagonal. Its optional limit is 0.00625, compared with 0.5 for congestion and 0.25 for the tie sweep. Individual base and scaling cases retain their manifest-specific parameters.

The frozen generator preserves legacy PCG64 streams and offsets; it does not derive every input from one common seed. Extended families use seed 20261024 with a group-key hash and input index; geometry and supplemental inputs use seed 20261030 with the full SHA-256 group-key digest and input index. The manifests and source snapshots retain the legacy seeds, stream offsets and exact construction rules. Input hashes identify the actual measured corpus.

Every assignment timing condition uses five frozen inputs, three rounds and a 20 ms target block. The estimate is the median of three round means, with equal weight to each input within a round. Single-matrix ratios divide reference and method estimates from the same successful paired attempt; they are not medians of input-wise ratios. Batch comparisons within one placement also use the same attempt. Worker-count contrasts compare matched inputs in separate process attempts under their respective CPU placements. Required cases use ordinary SciPy and optional cases use capped SciPy as their primary comparator. JVHYBRID's canonical coverage is required-only; its absence in optional panels is planned.

Ratios below 0.97 are losses, values in $[0.97, 1.03]$ are practical ties, and values above 1.03 are wins. This band is descriptive. We neither pool environments nor interpret timing-round spread as independent-session uncertainty. Derived tall/wide contrasts also join distinct matched-corpus case attempts. All estimates, references, input identities, attempts, status and reasons remain in the indexed supplement.

5.3 Geometry, representations and controlled policies

The tracking-like geometry design is inspired by Pylapy [19]. It draws r source points uniformly in the unit square, selects round(qr) survivors without replacement, and perturbs each surviving coordinate by

an independent zero-mean Gaussian with standard deviation $m\delta$, with motion multiplier $m \in \{0.25, 1\}$. It adds $c - \text{round}(qr)$ new uniform points, shuffles the observations, and uses Euclidean distances as costs. Moved points are not clipped or wrapped back into the square. Tracking-like geometry spans 8–4096 rows with square and 5 : 4 shapes. The size sweeps retain two settings at every size: high survival/low motion ($q = 0.9$, $m = 0.25$) and low survival/high motion ($q = 0.6$, $m = 1$), where $\delta = r^{-1/2}$ is the generator's characteristic spacing. The complete 256×320 and 1024×1280 panels retain all four survival/motion settings. The former also varies rejection limit and gate, including the primary cases deduplicated into that grid. Connections in the figures hold the generator, objective, limit policy, orientation and environment fixed apart from the named sweep variable.

In geometry, JVX, JVS and square-expanded SciPy are not planned above 1024 rows. This also stops unpadded required-square JVX/JVS. It is an additional light-profile rule, distinct from the general admission limit on padded buffers of 4096^2 entries. Missing points beyond a declared cutoff are not timing failures or evidence of a timeout. The native and matching compact SciPy comparisons continue to the largest sizes. Appendix E examines the protocol reduction using a separate saved full session, without adding its timings to the primary results.

The formulation and padded-layout panels separate mathematical representation from placement of real and dummy costs. The controlled policy panel uses separately built Rust and C++ variants with matched costs and output contracts. It varies bounded reduction budgets, bypasses reductions, disables free-column tie preference, and disables the required-only stall cutoff where applicable. Primary latency and separately instrumented work are joined by case, policy and input identity. Work-event counts are descriptive operations, not interchangeable units of processor time.

5.4 Validation, batch controls and memory

The saved campaign validates feasible outputs and their penalized objectives against the specified reference, checks represented costs and input hashes, and retains rejection cardinalities. Pair-array equality is not required when optimal solutions tie. Diagnostics have their own build and environment identities and are matched to primary input hashes. The saved preflight contains 4,195 passed fixture/method results, 2,864 unsupported combinations and 11 numerical-boundary limitation records per environment, with no blocking failures. The limitations occur in seven padded-layout methods; all 11 records concern a threshold-adjacent fixture or its transpose and have an exact represented-objective gap of 2^{-53}, although rounded totals agree. Unsupported fixtures and these boundary records are separate from the complete primary timing inventory. Primary input validation uses relative/absolute objective tolerances $10^{-11}/10^{-12}$. These checks support the sampled cases, not a proof of all floating-point transitions.

Batch contrasts distinguish Python loops, one-worker batch dispatch, four-worker execution, and controlled workspace lifetimes. Ratios are formed within each workload and environment; absolute batch times are retained separately. Repeated workspaces are tested directly rather than inferred from a single-call comparison. Independent problems must already be available together for inter-problem parallelism to apply.

Every memory condition has five fresh-process trials with resident input before baseline and a 50 ms output hold. Peak capture precedes reference validation. Linux records high-water RSS and Windows peak working set; both retain pre-call headroom below the process's previous peak. Additional peak, final resident growth and exact buffer arithmetic are different quantities. A zero additional high-water increment does not imply zero allocation. Memory orientation inputs are distribution-matched, whereas the timing orientation pairs use exactly transposed values.

5.5 Application and LSAPE contracts

The application suite reuses supplied 999-frame excerpts of MOT16-04 and MOT20-03 [25, 26], with supplied ground-truth boxes or cached RF-DETR Nano detections [27]. The detector cache used RF-DETR 1.10.1 at 768-pixel resolution and confidence 0.2. Centroid and SORT [28] come from pyppbox 4.0.0 [29, 30], pinned to source commit `f3f1e8e671d7`. Dataset/cache modifications and third-party notices are

Table 2: Methods, representations and controlled LSAPE arms. Here $r \le c$ denotes oriented JV dimensions; LSAPE retains original n, m. Definitions are not measured memory or speed claims.

Identity	Implementation / numerical contract	Representation or control
JVRUST; JVCPP	Rust; C++; float64 work for float32/64 storage	Required or uniform rejection; original costs plus linear state.
JVHYBRID	Rust with C++ square completion; canonical required coverage	Shares JVRUST's optional path; not separately timed there.
JVX; JVS	LAPX pairs API; square JV	Required square padding when needed; optional square augmentation.
CSGRAPH; LAPMOD	SciPy sparse matcher; LAPX sparse solver	Optional sparse adapter includes graph construction and output mapping.
SciPy required	Modified rectangular JV; float64	Original $r \times c$ required problem.
SciPy capped	Required solve on $\min(C, L)$, filter original $C < L$	$r \times c$; value is one-sided objective P.
SciPy dummy columns	Dense or diagonal dummy block costing L	$r \times (c + r)$; value P; each dummy has capacity one.
SciPy square	Equal endpoint penalties $L/2$	$(r + c)^2$; value $F = P + (c - r)L/2$.
Native rejection	Reusable search terminal of cost L	Original $r \times c$ costs; objective P.
LSAPE O / U	Unchanged LSAPE v4, full buffer	Equal endpoints / one-sided penalty.
LSAPE S / T	Specialized code, compact buffer	Ordered rejection scan / guarded cache.
LSAPE P	Specialized tracked code, full buffer	U-compatible leading dimension; matched-layout control.
Campaign boundary	Binary64 inputs, int64 pairs	Public-API checks, conversion, representation, solve and output mapping.
Application boundary	Float32 tracker costs, int32 pairs	Replay also widens captured values to float64 outside timing.
LSAPE boundary	Finite, nonnegative binary64 costs; int64 pairs	Validation, conversion, column-major copy, allocation, solve and output.

recorded in `benchmark/application/ATTRIBUTION.md`. Three rounds measure tracker updates and replay of the common captured SciPy matrices. Replay retains the float32 values and an exactly widened float64 view; it measures assignment calls separately from tracker updates. Detector inference, decoding and rendering are outside both boundaries. All eight APIs, including three distinct JVX return interfaces, remain in the supplement. The stock Centroid distance gate and SORT IoU post-filter and unique-feasible bypass are preserved; finite-limit assignment is not substituted into the trackers. The adapter normalizes application output to int32 pairs, unlike the controlled campaign's int64 pairs. Accuracy uses supplied-box motmetrics 1.4.0 [31] evaluation at IoU ≥ 0.5, without the official dataset class/ignore policy. Accuracy and prediction checks are reported separately from objective checks.

The Windows-only LSAPE study uses the July-2017 LSAPE-v4 Hungarian implementation [15], with five arms defined in Table 2. It retains 67 cases, four shared inputs, nine seeded-shuffled rounds with a 30 ms calibration target, and three fresh processes per Python environment. This is a frozen protocol rather than the assignment campaign's three-round estimator. Each process estimates a paired-round geometric-mean ratio; the reported central value and range are the median and observed minimum/maximum of the three process estimates. These ranges are descriptive, not confidence intervals.

With $h = \text{float64}(L/2)$ and $p = h + h$, equal endpoint costs (h, h) and one-sided costs $(p, 0)$ differ by $h(m - n)$; comparisons restore the same full objective. O/U changes formulation within unchanged upstream code, U/P measures specialization on the same full buffer and stride, S/T changes rejection tracking at fixed compact layout, and P/T changes padded versus compact tracked representation. The specialization also includes indexing and parameter-passing changes, so its components are not individually

isolated. All arms copy the real costs; compact storage removes only $n + m + 1$ doubles. The frozen bridge accepts finite nonnegative binary64 costs and limits bounded by DBL_MAX$/[16(n + m + 1)]$, with $(n + 1)(m + 1) \leq$ INT_MAX, round-to-nearest and gradual underflow. Signed costs, forbidden edges and required assignment are outside this experiment's contract. Guarded rejection caching falls back to the ordered scan outside its stated numerical range. This experiment introduces no JV row reductions and supplies no JV-versus-LSAPE ranking or measured memory benefit.

6 Results

6.1 Coverage and native comparisons

Figure 2 shows the full distribution of matching-SciPy/method ratios for native, legacy dense and sparse solvers on their planned single-matrix cases. Each curve is one solver and environment; its height is the fraction of that solver's admitted cases with a ratio at least as large as the horizontal coordinate. This retains losses as well as the extent of an advantage. All three native methods win all 102 required comparisons under the stated band. Across the four separately summarized environments, JVRUST's median ratio ranges from 3.67 to 4.66 for required assignment and 4.72 to 5.23 for optional assignment. These are medians over the selected cases within each environment, not pooled estimates. Tables 7 and 8 give the native family counts and exact non-win workloads.

For optional assignment, JVRUST's wins/ties/losses are 165/1/0, 165/1/0, 163/2/1 and 164/1/1 in L311/L314/W311/W314. JVCPP's corresponding counts are 163/2/1, 163/2/1, 165/1/0 and 166/0/0. The denominator is 166 capped-SciPy comparisons in every environment. All optional native ties and losses occur on congested-block inputs in three declared families: formulations, scaling and congestion. The only losses are on the hardest congestion-sweep case, case 041: JVRUST's ratios are approximately 0.984, 0.985, 0.916 and 0.926; JVCPP's are 0.956, 0.943, 1.023 and 1.036. The W311 Rust loss corresponds to about 9.2% extra latency relative to capped SciPy. The other exceptions are practical ties on the optional 256×320 formulation and 128×160 scaling inputs. Native rejection does not imply universal dominance over an original-dimension formulation.

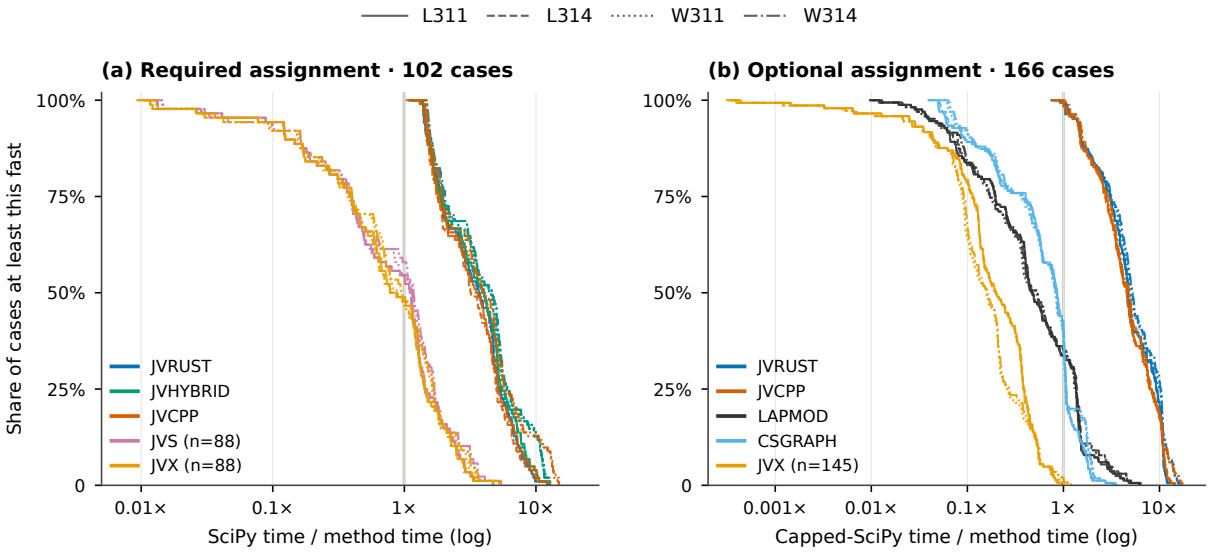


Figure 2: Complementary empirical distributions of matching-SciPy/method complete-call ratios. One curve represents one method and environment over that method's planned cases; reduced denominators are stated in the panel legends. Required and optional objectives remain separate. Color identifies method; solid, dashed, dotted and dash-dot lines identify L311, L314, W311 and W314, respectively. The vertical rule is parity and the gray band is $[0.97, 1.03]$. Environments are not averaged.

6.2 Geometry scaling

Figure 3 links absolute runtime and relative latency over the square geometry size sweep for $q = 0.9$ and motion standard deviation 0.25δ. Each line retains one method and environment. In this setting, CSGRAPH first exceeds a 3% advantage over capped SciPy at 512 rows in every environment; every sampled smaller size is a practical loss. The same crossover holds in the $q = 0.6$, motion-δ companion. Across both square settings and all environments, CSGRAPH takes approximately 15–20× the capped-SciPy latency at eight rows, including sparse construction. The supplement retains the second setting, 5 : 4 geometry and non-geometric scaling with the same line conventions. Appendix A gives the crossed geometry-sensitivity factors. No full-session observations or extrapolations fill planned exclusions.

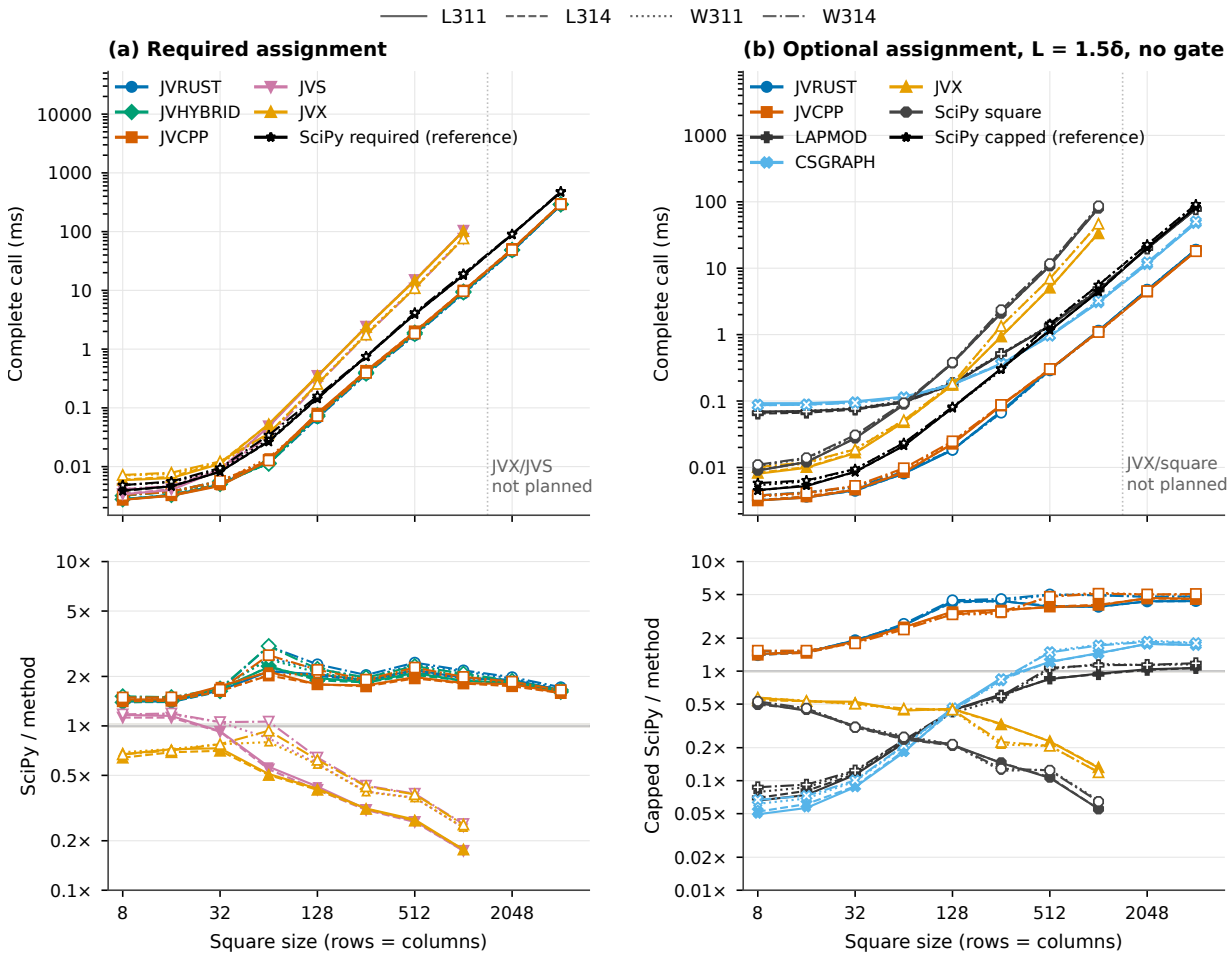


Figure 3: Square geometry with $q = 0.9$ and motion standard deviation 0.25δ: complete-call ms above, matching-SciPy/method ratios below. Required assignment and optional assignment with $L = 1.5\delta$ and no gate are separate columns. Method-specific colors and symbols combine with solid/dashed/dotted/dash-dot lines for L311/L314/W311/W314. JVX/JVS are not planned above 1024 rows in the required panel, including unpadded squares; optional JVX and square-expanded SciPy obey the same cutoff. No scaling exponent is fitted.

6.3 Representation and padded layout

The equivalent objectives in Section 3 do not require identical storage or work. Figure 4 shows five complete-call implementations on all 14 optional formulation workloads; the congested-block input is labelled because it contains the native near-parity exceptions. The supplement retains the diagonal-dummy variant and all 624 controlled layout conditions per environment. An expanded-versus-native gap includes construction and a different solver trajectory; it cannot be attributed entirely to padding allocation. Capped

SciPy controls dimensions while retaining different initialization, search and interface work. Constructor-only measurements and buffer dimensions remain secondary evidence.

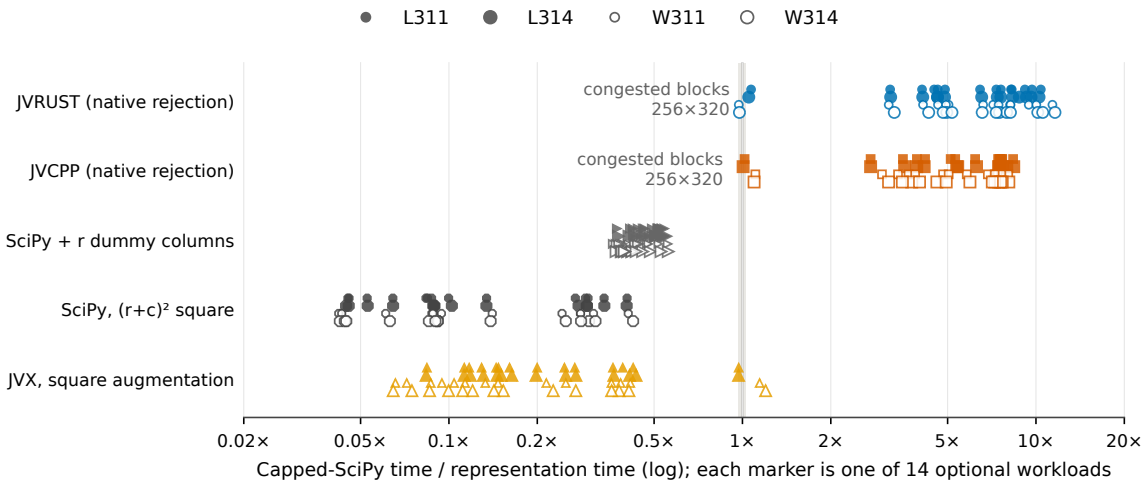


Figure 4: Distribution over the 14 optional formulation workloads for each displayed implementation, relative to capped SciPy. One marker is one workload/environment ratio; filled/open markers denote Linux/Windows, and smaller/larger markers denote Python 3.11/3.14. Deterministic offsets separate coincident observations without changing their horizontal values. The labelled congested-block workload retains the native practical ties. The diagonal-dummy and controlled-layout comparisons remain in the supplement.

6.4 Initialization policies and measured work

Figure 5 summarizes the distribution of case-normalized intervention/reference latency ratios for each controlled policy and environment. Median, interquartile range and extremes describe eligible workloads, not sampling uncertainty. Bypassing reductions is approximately 3.6–4.1× slower: median intervention/reference latency is 4.077, 4.024, 3.655 and 3.605 for Rust and 3.760, 3.689, 3.742 and 3.787 for C++, respectively. Other interventions retain case-level gains and regressions; a favorable median cannot select a universally optimal policy. The supplement retains all eligible per-case responses.

The counters in Table 3 show how initialization substitutes for later search. For the optional 256×320 base case, bypassing reductions changes mean initializer/search row scans from $512/0$ to $0/700.2$, while augmentations change from 0 to 256. For the tied required case, disabling free-column tie preference increases both initializer and search work. Conversely, disabling the required cutoff in the displayed geometry case increases initializer scans while reducing later search. These are controlled event changes; their relative costs still depend on access patterns and compiled code. Identical event vectors across languages and environments are printed once after verification, rather than averaged.

Table 3: Diagnostic mechanism anchors: mean events per matrix over five frozen inputs, reference → intervention. The verified event vectors agree across all four environments and Rust/C++; identical values are shown once. Zeros are literal counts. All six event fields and capacity records remain in the supplement.

Case; intervention	Init. scans	Search scans	Augmentations	Rejections
Optional uniform; bypass	512 → 0	0 → 700.2	0 → 256	0 → 0
Near square; budget 1	482.4 → 333.2	0 → 399.6	0 → 77.2	0 → 0
Tied base; free ties off	128 → 219.8	0 → 91.2	0 → 91	0 → 0
Geometry required; cutoff off	524.8 → 597.2	5.6 → 2.6	0.4 → 0.2	0 → 0
Optional blocks; budget 1	1285.8 → 384	872.4 → 1001.6	112 → 128	112 → 112

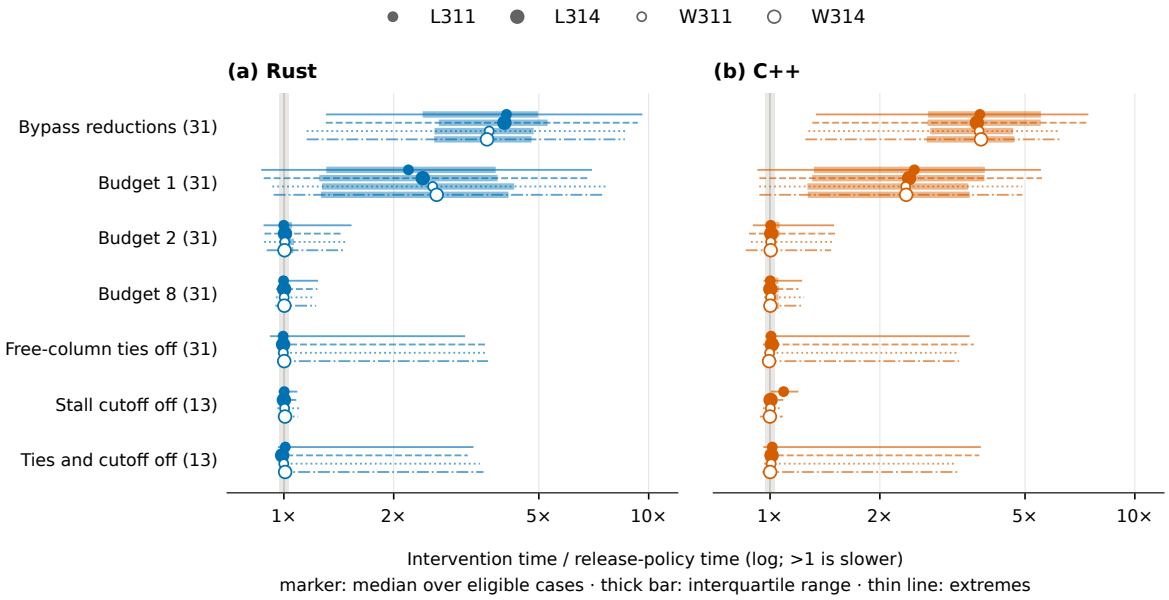


Figure 5: Policy distributions within each language and environment. For each intervention, a marker gives the median intervention/budget-4-reference latency ratio over eligible cases; the thick bar is the interquartile range and the thin line the observed minimum–maximum. These are descriptive spreads over different workloads, not confidence intervals. Filled/open markers denote Linux/Windows, and smaller/larger markers denote Python 3.11/3.14. Values above one mean a slower intervention. Table 3 uses separately instrumented calls.

6.5 Sparse methods against two references

Beating dense capped SciPy does not imply beating the native implementation. Figure 6 therefore compares each sparse method with both capped SciPy and JVRUST on every optional case. Sparse construction is included, so a solver can benefit from admissible sparsity yet lose once the complete interface is measured. CSGRAPH wins against capped SciPy on 53, 56, 47 and 55 of 166 cases, but against JVRUST on only 2, 2, 0 and 0. LAPMOD's corresponding win counts are 56, 55, 58 and 58 against capped SciPy and 2, 2, 3 and 3 against JVRUST. Retaining both axes exposes cases that a single dense-reference speedup would describe too favorably. Finite-after-gate diagnostics are not substituted for the actual below-limit admissible-edge density.

A post-hoc same-OS comparison identifies seven case/environment events where the primary SciPy reference latency is at least 30% larger in one Python environment while JVRUST differs by at most 10%; input hashes agree. Observed reference ratios span 1.404–1.690. Within these events, three CSGRAPH classifications differ between sibling environments. Holding the target CSGRAPH latency fixed and substituting only its sibling reference changes two classifications, not three. Native classifications do not change under either comparison. These are sensitivity checks, not corrected measurements or evidence that reference timing caused all observed differences.

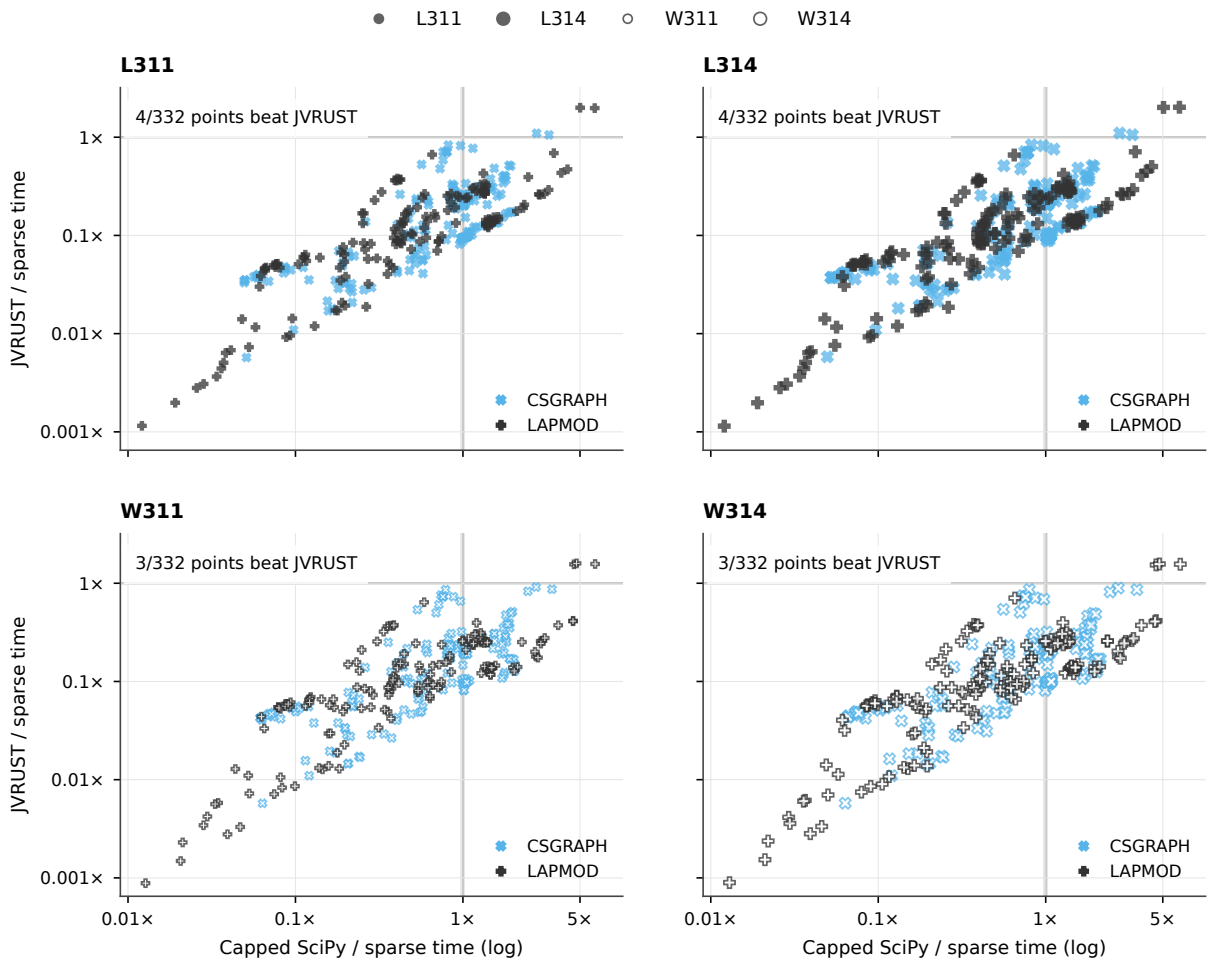


Figure 6: All 166 optional cases for each sparse method, with one panel per environment. One point is one case/method: its horizontal coordinate is capped-SciPy/sparse and its vertical coordinate JVRUST/sparse complete-call latency. Method colors distinguish CSGRAPH and LAPMOD. Parity rules and practical-tie bands apply on both log axes. Beating the dense reference does not imply beating JVRUST.

6.6 Ordered cost structure and rejection

Figure 7 shows the ordered congestion, forbidden-entry and quantization sweeps. Required feasibility is planted in the forbidden-entry sweep. Changing the congested row fraction changes measured latency at fixed dimensions; the sweep has no counters separating initialization from completion work. Its 90% congestion endpoint, case 041, contains the only optional native losses: Windows JVRUST and Linux JVCPP. The other congested-block exceptions occur in the formulation and scaling families (Table 8). The quantized-cost sweep measures sensitivity to tie density; the separate policy intervention and counters test free-column preference.

The rejection-limit appendix links complete-call latency to realized unmatched-row fractions read from the saved validation records. A limit changes the objective and search prices, so this is not a causal experiment on rejection fraction alone. In particular, equal realized cardinality need not imply equal search work or latency.

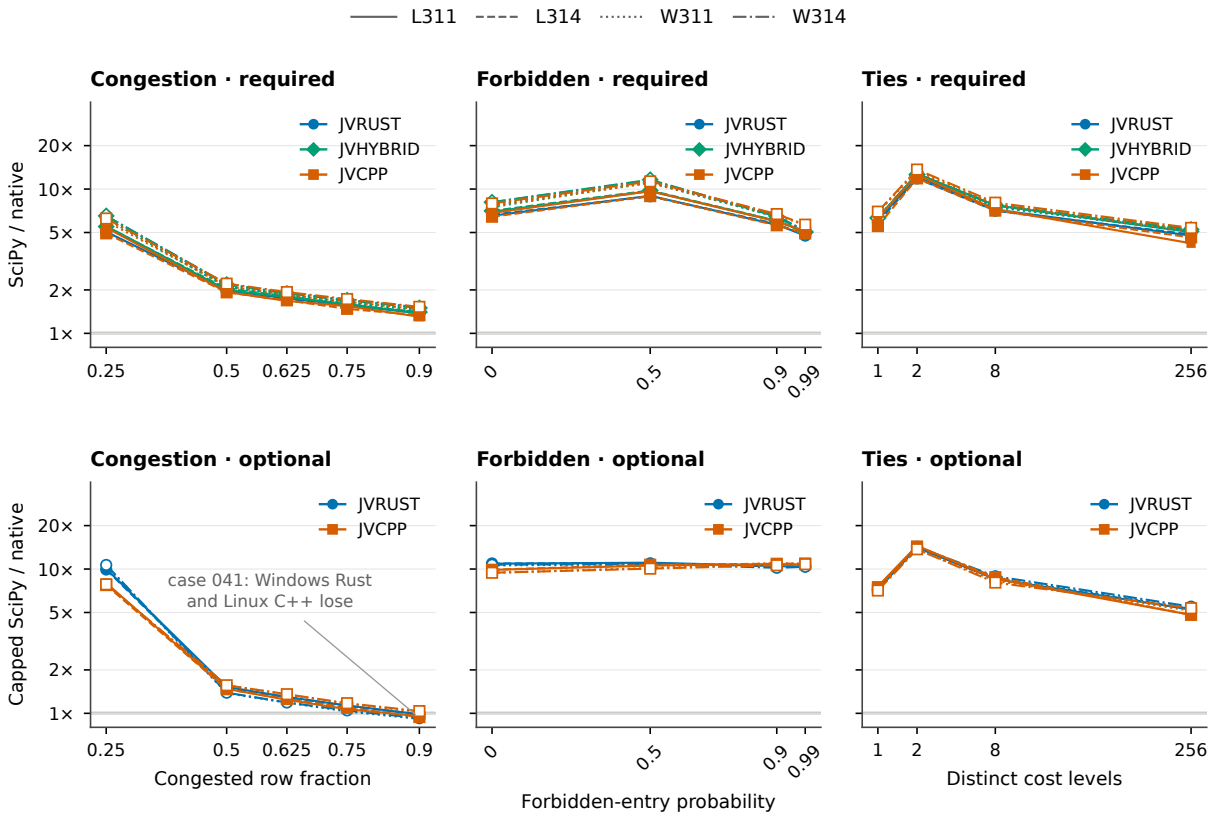


Figure 7: Ordered native-solver sweeps at 256×320. Columns vary congested row fraction, forbidden-entry probability and quantization levels; rows separate required and optional assignment. One curve is one method/environment, with all other case parameters fixed. Color identifies method; solid/dashed/dotted/dash-dot lines identify L311/L314/W311/W314. Ratios above one favor the native method over required or capped SciPy. The case-041 annotation retains both Rust/Windows and C++/Linux losses.

6.7 Batching and workspace lifetime

Figure 8 separates loop-to-batch dispatch, worker count and workspace lifetime. A one-worker batch can reduce repeated interface work without introducing parallelism; a four-worker contrast additionally changes scheduling and worker-local workspace use. The workspace controls test retention directly, including effects that can differ for tall inputs. Every declared workload is retained, including near-one results. The supplement also preserves the redundant total-worker-lifetime contrast. Retention is not uniformly beneficial: the L311 C++ four-worker optional $8 \times 8192 \times 256$ workload has a recreated/retained solver-state ratio of about 0.828. Worker-count changes can also lose: the tiny JVX batch has one/four-worker ratios of 0.489, 0.547, 0.389 and 0.447 in L311/L314/W311/W314. The $64 \times 64 \times 80$ JVX batch also loses in L311, W311 and W314 (approximately 0.931, 0.579 and 0.599), while L314 is a practical tie. These losses prevent interpreting a batch API or retained workspace as an unconditional speedup. These controlled contrasts support decisions for independent batches but do not promise the same gains for a tracker that presents one matrix at a time.

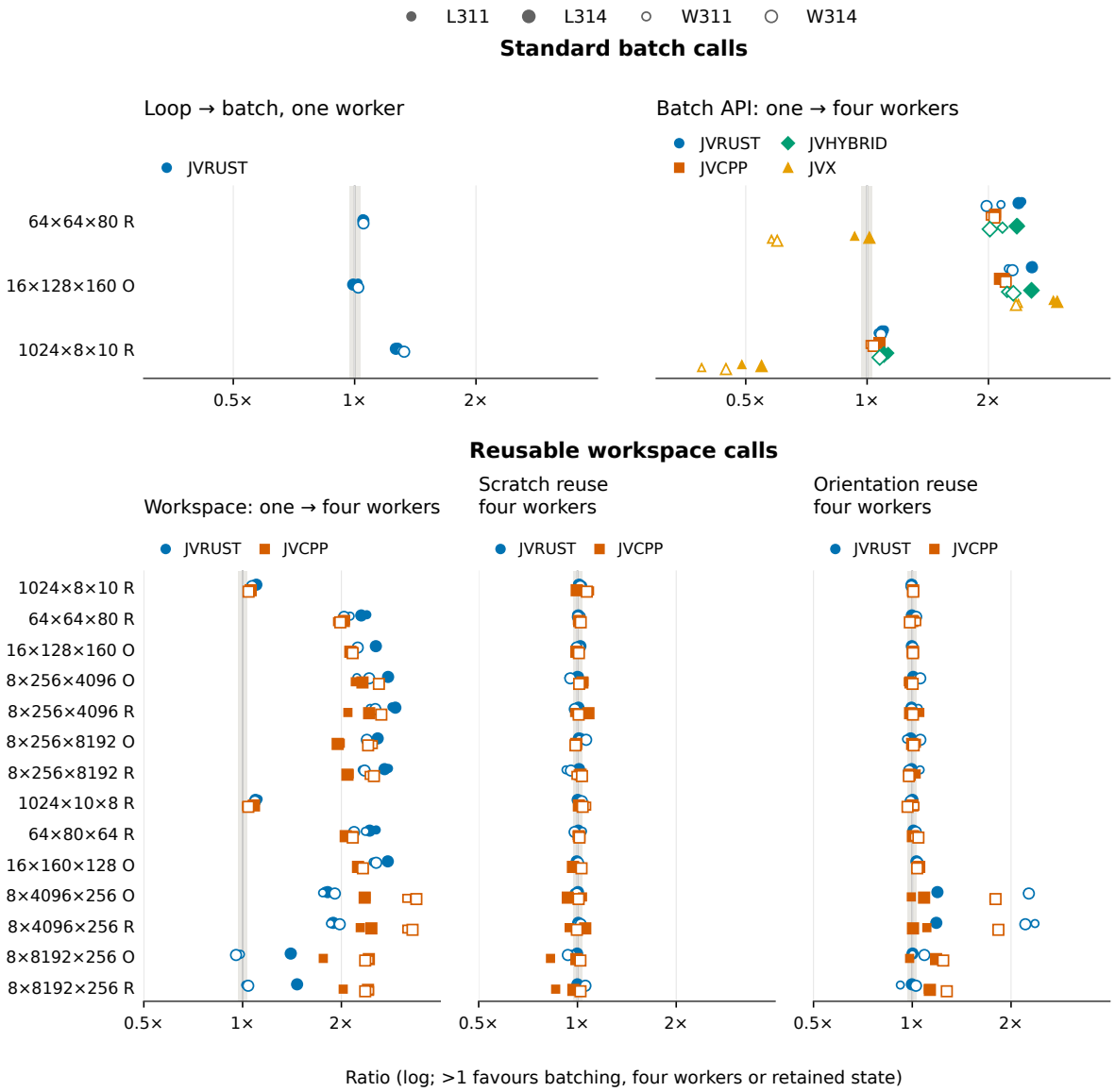


Figure 8: Named batch contrasts with workload rows. Each marker is a method/environment ratio: loop/one-worker batch, one/four workers, recreated/retained scratch ($W1/W0$), or recreated/retained orientation storage ($W2/W1$), as labelled. Colors identify methods; filled/open markers denote Linux/Windows, and smaller/larger markers denote Python 3.11/3.14. The workspace panel uses W0 for worker scaling and four workers for reuse. All three standard and 14 workspace workloads remain visible; one-worker reuse and the remaining contrasts are supplementary. Values below one retain regressions.

6.8 Applications: replay, updates and output differences

Figure 9 shows how assignment-replay gains change at the tracker-update boundary within each pipeline. Its native and legacy API groups use observed ranges across their APIs and the four environments, not averaged timings. Each native method has four update wins and four practical ties in each environment: Centroid gains attenuate relative to replay, while SORT updates remain near parity. This measures cached-input tracker updates, not detector-plus-tracker frame rate. The supplementary per-API table retains every environment-specific replay/update pair, including slower outcomes.

Each environment records 135,702 replay validation checks on 7,539 matrices, covering the eight timed APIs plus the Rust alias and two precision views. All stored nonzero objective-gap counts and mismatch counts are zero. Replay checks and tracking checks answer different questions. Equal objective values on the captured common matrices do not guarantee identical choices under ties, and altered choices can change

later tracker inputs. The saved native predictions differ from SciPy on some Centroid pipelines, including both favorable metric changes and additional identity switches. Appendix D retains the workload counts, accuracy deltas and prediction identities; no general accuracy improvement is claimed.

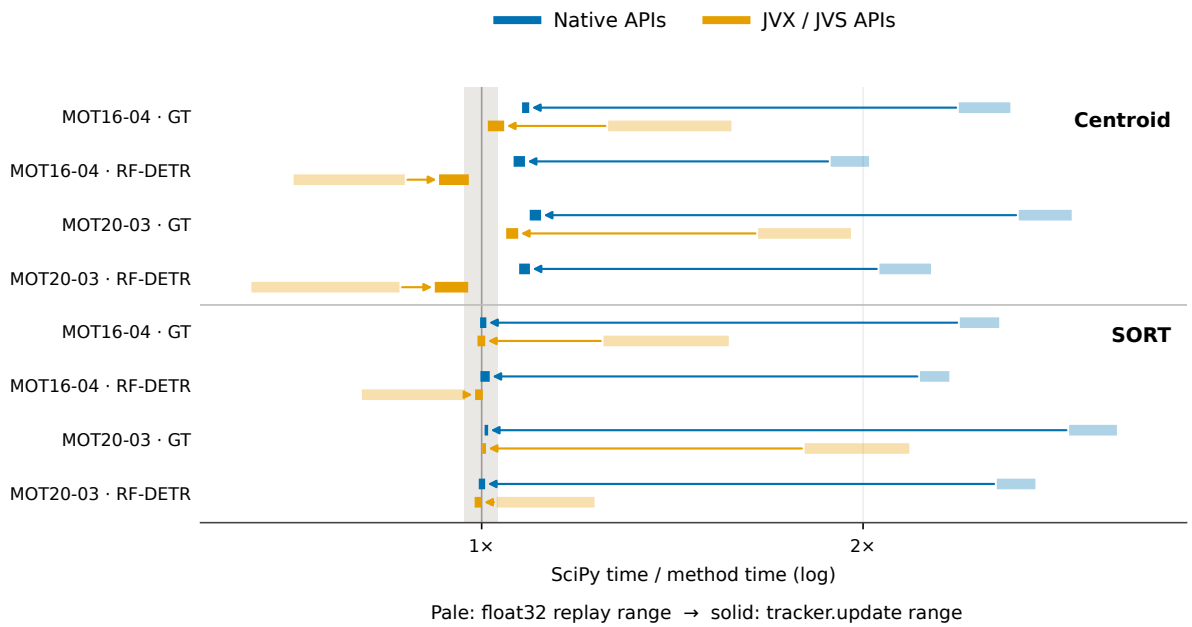


Figure 9: Replay-to-update changes within eight pipelines. Pale segments span float32 replay ratios and solid segments span cached tracker-update ratios; arrows connect the nearest range boundaries in that direction. Native ranges contain 12 API/environment observations (three APIs, four environments); legacy ranges contain 16 (three JVX interfaces and JVS, four environments). These envelopes are neither means nor uncertainty intervals. Blue identifies the native group and amber the legacy group. All 224 individual API/pipeline/environment pairs remain in the supplementary table.

6.9 Controlled LSAPE results

Figure 10 shows all 67 case estimates for each of five contrasts, separately for the two Windows interpreters. Table 9 retains the per-family counts. Formulation (O/U), matched-buffer specialization (U/P), rejection tracking (S/T), and layout (P/T) answer distinct questions. Their estimates should not be multiplied from separately summarized medians. Regressions remain as relevant as gains, and a ratio inside the practical band does not establish equivalence. Matched-buffer specialization U/P wins all 67 cases in each Windows environment. Formulation-only O/U yields wins/ties/losses of 20/5/42 in W311 and 20/9/38 in W314. Compact layout P/T yields 0/43/24 and 0/38/29; rejection tracking S/T yields 0/66/1 and 0/67/0. The declared primary contrast in the frozen manifest, O/P, yields 38/12/17 and 38/13/16. Compact layout has no improvement exceeding 3% and is slower beyond the band in 24 and 29 cases, with minimum P/T ratios of approximately 0.723 and 0.720. Formulation alone also loses more often than it wins. Rejection tracking is almost entirely within the practical band, with one W311 loss. The specialization advantage therefore does not imply that each transformation helps. The full export also retains absolute arm times and all eight saved comparisons, including total contrasts.

These results characterize the Hungarian adaptation and its measurement boundary. They do not identify the source of the JV/SciPy gap, reproduce the 2016 publication's timings, or establish a general benefit from rejection-slack caching. Process ranges describe the recorded same-host runs rather than independent-day variability.

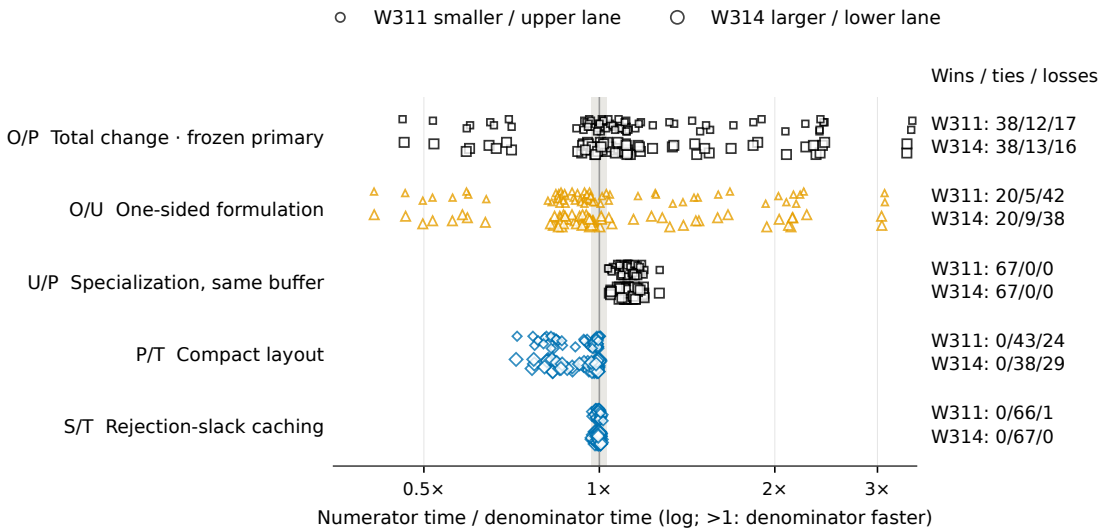


Figure 10: All 67 LSAPE cases in each contrast and Windows environment, with the frozen primary O/P first. Each point is the median of three process estimates for one case; smaller/larger open markers denote W311/W314 in separately keyed lanes. Color and shape identify the denominator arm: P black squares, U amber triangles and T blue diamonds. Deterministic vertical spread separates points without changing ratios. Counts beside each row use unrounded values and the practical band. Ratios above one favor the denominator arm. The complete export retains process ranges and all eight saved contrasts.

6.10 Orientation sensitivity

The orientation pairs hold generated values and objective fixed while transposing the original matrix. Figure 11 gives tall/wide complete-call latency, retaining every method planned in both orientations. Ratios are formed from two case attempts within one environment, not from same-attempt paired observations. Input transposition is outside timing, while any internal orientation copy, solve and output handling remain inside it.

For the optional 64×16384 pair, JVRUST tall/wide ratios are approximately 3.250, 3.327, 3.256 and 3.266; JVCPP ratios are 10.681, 10.647, 16.827 and 17.058 in the standard environment order. CSGRAPH shows an even larger orientation response, while LAPMOD is much closer to parity on this pair. These differences motivate measuring the actual input orientation. They do not isolate a transpose-copy cost, a language effect or an operating-system effect.

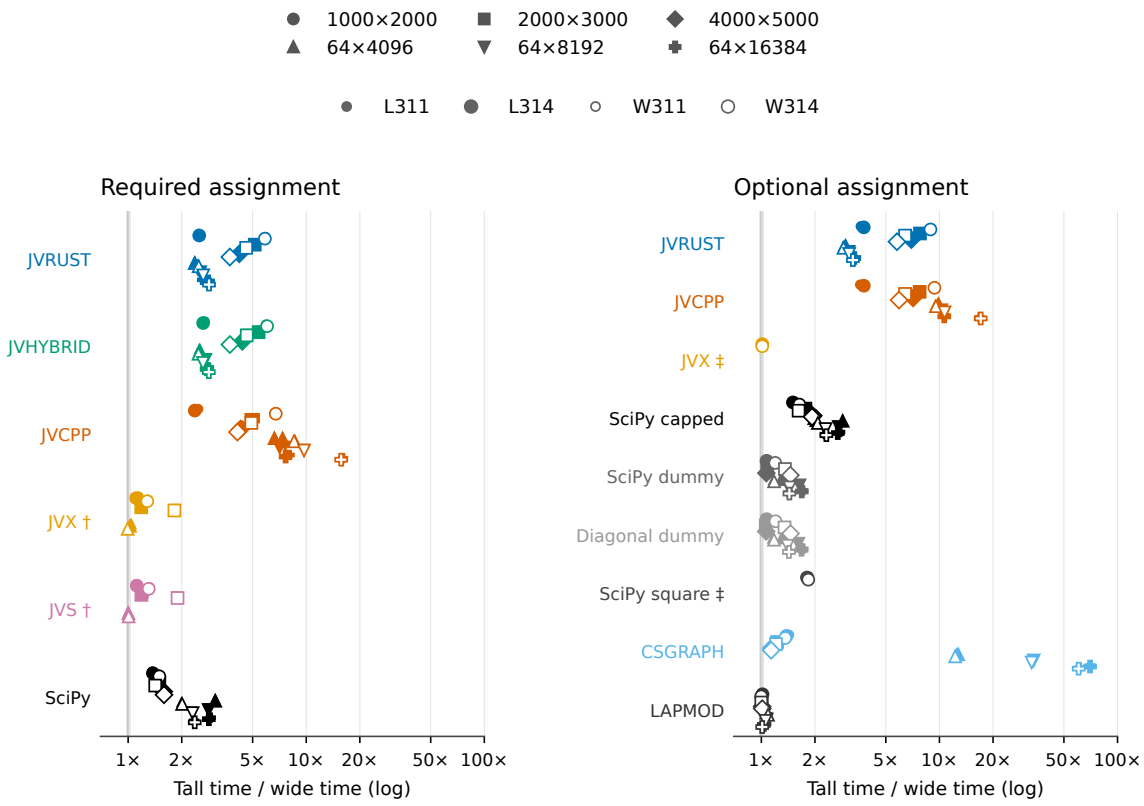


Figure 11: Tall/wide complete-call ratios by method, with required and optional objectives separated. One marker is one of six matched shape pairs in one environment; shape symbols distinguish the pairs and colors identify methods. Filled/open markers denote Linux/Windows, and smaller/larger markers denote Python 3.11/3.14. Ratios above one mean a slower tall call. All 74 planned method-pair comparisons per environment are retained, with unavailable shape/method pairs identified as not planned. Wide and tall timings come from distinct attempts with matched underlying values.

6.11 Memory endpoints and counter visibility

Table 4 retains both orientations and every planned method at the extended 256×4096 endpoints. Linux has a zero median additional high-water increment in 84 of 91 conditions for each interpreter; Windows has 18 and 17. These zeros cannot rank intrinsic workspace requirements across operating systems. Appendix C compares each raw trial's high-water increment with its net post-call resident growth and retains the baseline headroom that can conceal an increment. Buffer arithmetic remains distinct: optional square expansion needs 144.5 MiB of float64 costs for this 8 MiB original problem, irrespective of the counter's visibility.

Table 4: Extended memory endpoints in MiB, median [minimum, maximum] over five fresh processes. Linux and Windows counter groups remain separate. A zero high-water increment means no increase above an earlier process peak, not zero allocation. Cost MiB is the arithmetic size of one binary64 representation, not additional allocation; native calls can borrow or internally orient the input. The companion export retains condition-specific headroom. R/O denote required/optional; 0^* means no new high-water peak.

			Linux: `ru_maxrss`		Windows: peak working set	
Shape/objective	Method	Cost MiB	L311	L314	W311	W314
256×4096 / R	JVRUST	8	0*	0*	0*	0*
			[0,0]	[0,0]	[0,0]	[0,0]
	JVCPP	8	0*	0*	0*	0*
			[0,0]	[0,0]	[0,0]	[0,0]
	SciPy req.	8	0*	0*	5.17	5.15
			[0,0]	[0,0]	[5.15,5.18]	[5.14,5.16]
	JVX	128	29.11	35.98	125.07	125.05
			[29.00,29.54]	[35.74,36.34]	[125.03,125.08]	[125.04,125.05]
256×4096 / O	JVRUST	8	0*	0*	0*	0*
			[0,0]	[0,0]	[0,0]	[0,0]
	JVCPP	8	0*	0*	0*	0*
			[0,0]	[0,0]	[0,0]	[0,0]
	SciPy capped	8	0*	0*	5.16	5.16
			[0,0]	[0,0]	[5.15,5.18]	[5.14,5.18]
	SciPy dummy	8.5	0*	0*	5.70	5.69
			[0,0]	[0,0]	[5.68,5.70]	[5.67,5.73]
	SciPy square	144.5	83.92	95.77	141.83	141.80
			[83.65,84.09]	[95.52,95.90]	[141.81,141.83]	[141.80,141.82]
4096×256 / R	JVRUST	8	0*	0*	5.03	5.05
			[0,0]	[0,0]	[5.03,5.07]	[5.04,5.09]
	JVCPP	8	0*	0*	5.04	5.05
			[0,0]	[0,0]	[5.04,5.05]	[5.05,5.18]
	SciPy req.	8	0*	0*	13.18	13.16
			[0,0]	[0,0]	[13.16,13.19]	[13.14,13.19]
	JVX	128	28.52	35.86	125.03	125.05
			[28.45,28.83]	[35.78,36.27]	[125.03,125.14]	[125.04,125.13]
4096×256 / O	JVRUST	8	0*	0*	5.07	5.09
			[0,0]	[0,0]	[5.07,5.07]	[5.08,5.18]
	JVCPP	8	0*	0*	5.07	5.08
			[0,0]	[0,0]	[5.07,5.11]	[5.08,5.21]
	SciPy capped	8	0*	0*	13.18	13.17
			[0,0]	[0,0]	[13.16,13.19]	[13.15,13.19]
	SciPy dummy	8.5	0*	0*	5.70	5.69
			[0,0]	[0,0]	[5.70,5.70]	[5.66,5.70]
	SciPy square	144.5	81.94	94.71	141.84	141.79
			[81.22,82.43]	[94.31,95.63]	[141.82,141.84]	[141.77,141.82]

7 Discussion

The native implementations combine established initialized rectangular JV and uniform-rejection ideas in usable interfaces. Their required and optional results support a practical advantage over the specified compact SciPy references on most measured cases, with congested-block exceptions retained. The capped reference is essential: comparing only with square expansion would conflate avoiding dummy costs with the remaining implementation differences.

The controls make the interpretation more specific. Representation and layout panels separate alternative buffers; policy variants change individual initialization choices within matched builds; counters expose a shift between initializer and completion work; batch controls distinguish dispatch, workers and retention; and orientation pairs reveal different complete-call responses to the same costs. None of these comparisons assigns the entire native/SciPy difference to one component. The sparse comparisons also show why a favorable ratio against one baseline is insufficient to establish a solver ranking.

JVRUST and JVHYBRID share the rectangular Rust core, so differences there cannot establish a different rectangular algorithm. Their required-square comparison changes the completion kernel within the shared design. JVCPP changes more of the implementation and build. The observed differences therefore describe implementations, not an intrinsic Rust-versus-C++ effect. Likewise, matched package versions and verified placement improve environmental control without turning separate operating-system and interpreter sessions into causal experiments.

For applications, the assignment objective, tie convention, represented precision and output interface must match the intended contract. Replay isolates common matrix calls but is not the evolving tracker workload after a different matching decision. Tracker updates include work beyond assignment, and detector inference adds another boundary. The observed attenuation and prediction differences justify benchmarking the actual pipeline rather than extrapolating an isolated solver speedup.

8 Limitations and threats to validity

There is one primary session per environment, on one recorded processor model. The study does not estimate variation across days, machines or compilers. Frequency, power-management behavior and background activity remain uncontrolled. Three rounds over five frozen inputs give descriptive within-session estimates, not population confidence intervals or significance tests. Within-attempt rounds cannot detect a persistent attempt-level shift. The seven post-hoc reference discrepancies in Section 6.5 show why reference stability matters, but interpreter, build and collection session change together; the data cannot identify a causal attempt effect. All primary observations and classifications are retained. The practical tie band is a reporting convention, and a tie does not prove numerical or performance equivalence.

The initialization heuristic was selected using related workload families. Controlled ablations characterize the selected panel, but do not establish a held-out optimum or universal policy. The synthetic geometry is tracking-like rather than a sample from the application corpus. The light profile deliberately omits two geometry settings outside the complete panels and stops selected baselines above 1024 rows. No missing observation is extrapolated. The separate full-session sensitivity check supports a bounded reporting choice, not statistical equivalence of protocols.

The proofs hold in exact arithmetic. Float64 working state cannot recover distinctions lost during cost conversion and does not prove arbitrary-range floating-point optimality. Equal recorded floating-point objective totals can conceal smaller exact differences. Pair identities and cardinality can differ among equal-cost optima, especially at the rejection limit. Saved validation supports the actual sampled inputs and represented objectives, not every code transition or error path.

The applications cover two cached clips, two detection sources and two trackers. Detector preparation is input provenance, not a newly measured stage. Replay gains and tracker-update gains cannot establish a general full-pipeline speedup, and mixed tracking metric changes cannot establish an accuracy benefit. The

frozen LSAPE contract is narrower than the JV interface and its process replications remain Windows-only. It is not a direct comparison with JV or every compact error-correcting algorithm.

Memory counters have platform-specific semantics and prior-peak headroom. Neither a zero increment nor final resident growth measures total allocation. The trials do not establish cross-platform memory efficiency or long-running pool retention. Constructor timings, instrumented events and primary complete-call timings have different boundaries and must not be interchanged.

9 Conclusion

Native rectangular rejection integrates established assignment principles without requiring square expansion. In each of four environments, the measured Rust, hybrid and C++ implementations win all 102 required comparisons against rectangular SciPy under a 3% practical band. JVRUST's median reference/method ratio ranges from 3.67 to 4.66 for required assignment and 4.72 to 5.23 for optional assignment, with each environment summarized separately. Optional gains remain workload-dependent: congested-block inputs produce ties and losses, including 9.2% extra JVRUST latency in the worst measured Windows comparison with capped SciPy. The controls show why representation, initialization, input structure, orientation and interface work all matter when selecting an implementation. Application replay gains attenuate in tracker updates, while the LSAPE study remains a separate within-Hungarian experiment. The saved-data figures and indexed evidence support these descriptive results without implying algorithmic priority, universal superiority or unmeasured application benefits.

Data and software availability

The repository for this work is github.com/rathaROG/lap-paper. Publication of the repository update and arXiv replacement accompanying this revised manuscript is pending; the public versions currently describe the earlier study. The source and evidence described below accompany this local revision.

The primary data are the four `paper-run-light` roots under `benchmark/runs/`: `linux-py3.11.18-20261031-0455`, `linux-py3.14.10-20261031-0630`, `windows-py3.11.18-20261030-2322` and `windows-py3.14.10-20261031-0322`. The accompanying `benchmark/evidence/` bundle packages the files needed for regeneration, together with all primary raw timing/validation logs, in five checksummed archives. Its manifest identifies each file, digest and source root; the fifth archive supports only the separate full-session sensitivity analysis. These are saved observations and metadata, not executable benchmark builds. Native module hashes and controlled build identities distinguish release interfaces from policy and diagnostic variants.

After installing `benchmark/requirements-artifacts.txt`, run `python benchmark/paper_evidence.py extract` to restore and verify the saved files. Then `python benchmark/generate_paper_artifacts.py` from the repository root regenerates the empirical figures, table fragments and indexed supplement using saved observations only. Outputs are under `latex/images/current/`, `latex/generated/` and `benchmark/paper-artifacts/`. The companion `latex/supplementary_figures.tex` builds the paginated figure atlas; its output is `output/pdf/supplementary_figures.pdf`. The separate `linux-py3.11.18-20261030-1434/paper-run` session is used only for the supplementary protocol sensitivity; none of its timings enters the four-environment plots. No solver execution is required to reproduce the reported projections.

Acknowledgements

The implementations retain attribution to Tomas Kazmar's lap, Vadim Markovtsev's src-d/lapjv, and the contributors acknowledged in the source distributions. The assignment and error-correction literature provides the theoretical background and comparison points for this implementation study.

AI assistance disclosure

The first author led the research and proposed the approach studied in this paper. AI coding assistants (OpenAI Codex with GPT-6 Astra and GPT-6.1 Sol, and Anthropic Claude Code with Claude Opus 5.5) assisted under the authors' directions with source inspection, experiment tooling, evidence checks, figure generation and manuscript drafting. Saved artifacts identify the computations supporting the results. Responsibility for reviewing the scientific claims, attribution and final text remains with the authors.

References

[1] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022. URL https://arxiv.org/abs/2110.06864.

[2] Tomas Kazmar et al. lap. Source implementation. URL https://github.com/gatagat/lap. Evaluated through its LAPX adaptation.

[3] R. Jonker and A. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38:325–340, 1987. doi: 10.1007/BF02278710.

[4] A. Volgenant. Linear and semi-assignment problems: A core oriented approach. *Computers & Operations Research*, 23(10):917–932, 1996. doi: 10.1016/0305-0548(96)00010-X.

[5] J. Bijsterbosch and A. Volgenant. Solving the rectangular assignment problem and applications. *Annals of Operations Research*, 181:443–462, 2010. doi: 10.1007/s10479-010-0757-3.

[6] Russ J. Vander Wiel and Nikolaos V. Sahinidis. The assignment problem with external interactions. *Networks*, 30 (3):171–185, 1997. doi: 10.1002/(SICI)1097-0037(199710)30:3<171::AID-NET3>3.0.CO;2-I.

[7] David F. Crouse. On implementing 2d rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52(4):1679–1696, 2016. doi: 10.1109/TAES.2016.140952.

[8] Sébastien Bougleux and Luc Brun. Linear sum assignment with edition. Technical report, Normandie Université, 2016. URL https://arxiv.org/abs/1603.04380.

[9] William Jones, Aziem Chawdhary, and Andy King. Optimising the Volgenant–Jonker algorithm for approximating graph edit distance. *Pattern Recognition Letters*, 87:47–54, 2017. doi: 10.1016/j.patrec.2016.07.024. URL https://kar.kent.ac.uk/56765/1/main.pdf.

[10] Sébastien Bougleux, Benoit Gaüzère, David B. Blumenthal, and Luc Brun. Fast linear sum assignment with error-correction and no cost constraints. *Pattern Recognition Letters*, 134:37–45, 2020. doi: 10.1016/j.patrec.2018.03.032. First published online in 2018.

[11] Ratha SIV. LAPX: Linear assignment problem solvers. Version 0.13.0, 2026. URL https://github.com/rathaROG/lapx.

[12] Vadim Markovtsev et al. lapjv. Source implementation. URL https://github.com/src-d/lapjv. Accessed 2026-10-22.

[13] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[14] SciPy developers. Documentation for scipy.optimize.linear_sum_assignment, . URL https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html. Accessed 2026-10-22; experiments use SciPy 1.17.1.

[15] Sébastien Bougleux, Luc Brun, et al. LSAPE bipartite matching toolbox. Version 4, July 2017, 2017. URL https://bougleux.users.greyc.fr/lsape/.

[16] The MathWorks, Inc. assignjv, assignmunkres, and assignauction. Sensor Fusion and Tracking Toolbox documentation, introduced in R2018b. URL https://www.mathworks.com/help/fusion/ref/assignjv.html. Accessed 22 October 2026.

[17] David F. Crouse. The Tracker Component Library: Free routines for rapid prototyping. *IEEE Aerospace and Electronic Systems Magazine*, 32(5):18–27, 2017. doi: 10.1109/MAES.2017.160215. Routines `assign2DMissedDetect` and `assign2DFull`, https://github.com/USNavalResearchLaboratory/TrackerComponentLibrary, accessed 22 October 2026.

[18] SciPy developers. Documentation for scipy.sparse.csgraph.min_weight_full_bipartite_matching, . URL https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.min_weight_full_bipartite_matching.html. Accessed 2026-10-31; experiments use SciPy 1.17.1.

[19] Raphael Reme. Pylapy: Linear assignment problem solvers in python. Version 1.2.0, 2026. URL https://github.com/raphaelreme/pylapy. Upstream project, MIT license; accessed 31 October 2026. Adapted layout revision is recorded in the benchmark source.

[20] Ratha SIV. laprust: Native Rust linear assignment solvers with implicit rejection and batch solving. Version 1.2.0, 2026. URL https://github.com/rathaROG/laprust.

[21] Ratha SIV. lapcpp: Native C++ linear assignment solvers with implicit rejection and batch solving. Version 1.2.0, 2026. URL https://github.com/rathaROG/lapcpp.

[22] Ratha SIV and contributors. Float32 arithmetic can return a suboptimal assignment. src-d/lapjv issue 94, 2026. URL https://github.com/src-d/lapjv/issues/94. Opened 28 October 2026; accessed 29 October 2026.

[23] SciPy developers. SciPy release metadata on PyPI, 2026. URL https://pypi.org/project/scipy/1.18.1/. Release catalogue and Python requirements checked 31 October 2026; versions 1.17.1 and 1.18.1.

[24] SciPy developers. SciPy tagged assignment sources, 2026. URL https://github.com/scipy/scipy/tree/v1.18.1/scipy. Compared with tag v1.17.1: optimize/rectangular_lsap/rectangular_lsap.cpp and .h; sparse/csgraph/_matching.pyx. Checked 31 October 2026.

[25] Anton Milan, Laura Leal-Taixé, Ian Reid, Stefan Roth, and Konrad Schindler. MOT16: A benchmark for multi-object tracking. arXiv:1603.00831, 2016. URL https://arxiv.org/abs/1603.00831.

[26] Patrick Dendorfer, Hamid Rezatofighi, Anton Milan, Javen Shi, Daniel Cremers, Ian Reid, Stefan Roth, Konrad Schindler, and Laura Leal-Taixé. MOT20: A benchmark for multi object tracking in crowded scenes. arXiv:2003.09003, 2020. URL https://arxiv.org/abs/2003.09003.

[27] Isaac Robinson, Peter Robicheaux, and Matvei Popov. RF-DETR. Roboflow software, 2025. URL https://github.com/roboflow/rf-detr. Software citation metadata from CITATION.cff; first released 20 March 2025. Evaluated package version 1.10.1; accessed 27 October 2026.

[28] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003. URL https://arxiv.org/abs/1602.00763.

[29] Ratha SIV. pyppbox: Toolbox for people detecting, tracking, and re-identifying. Version 4.0.0, 2026. URL https://github.com/rathaumons/pyppbox. Source commit f3f1e8e671d7cad7e0df2f2469ae0f5ee6c27258; accessed 27 October 2026.

[30] Ratha Siv, Matei Mancas, Bernard Gosselin, Dona Valy, and Sokchenda Sreng. People tracking and re-identifying in distributed contexts: Extension study of PoseTReID. In *2022 9th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*, pages 337–342, Jakarta, Indonesia, 2022. IEEE. doi: 10.23919/EECSI56542.2022.9946587.

[31] Christoph Heindl, Jack Valmadre, and contributors. py-motmetrics: Metrics for multiple object tracker benchmarking. Version 1.4.0, 2022. URL https://github.com/cheind/py-motmetrics/tree/v1.4.0. Software; accessed 27 October 2026.

A Scaling and geometric sensitivity

The supplementary figures use family-wise line panels for the 79 non-geometric scaling, large-square, large-rectangle and extreme-rectangle cases, alongside the 5 : 4 and second-setting geometry companions. Each curve holds objective, generator, orientation, limit policy, method and environment fixed apart from its declared sweep variable. Per-case policy responses and controlled layouts remain separate panels. Exact values for all 268 single-matrix cases remain in the indexed exports.

Figure 12 uses a heatmap for the crossed 256×320 geometry factors: four survival/motion settings, three gates and three limits, with environments as separate rows. Cells use capped-SciPy/method ratios. This is the only heatmap; it shows the crossed-factor comparisons directly rather than joining different settings as a size trend.

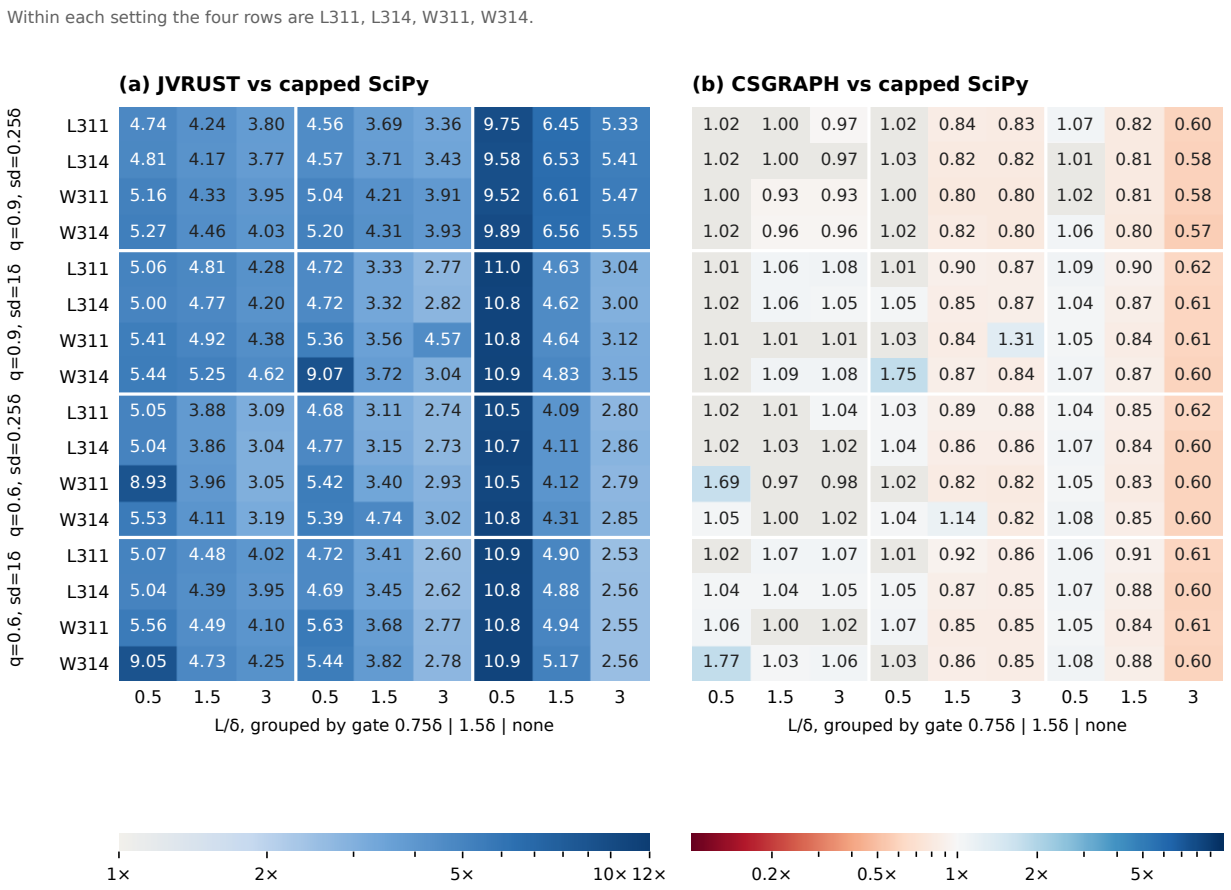


Figure 12: Geometry sensitivity at 256×320: JVRUST and CSGRAPH relative to capped SciPy. Columns cross gate and L/δ; row groups cross survival and motion, retaining L311/L314/W311/W314 separately within each group. Each printed cell is one observed complete-call ratio. The color key is logarithmic, and classification uses unrounded values. Repeated or reference-sensitive cells remain recorded observations, not averaged or corrected measurements.

B Rejection-limit sweep

The measured limit sweep in Figure 13 reuses saved costs within each family and reads realized cardinality from validation records. No matrix is regenerated and no solver is rerun to obtain the plotted unmatched fractions. In the congested-block 256×320 case, every saved input has 144 matches at both $L = 0.05$ and $L = 0.5$, leaving 43.75% of smaller-side rows unmatched. JVRUST latency nevertheless changes from 68.94 to 706.63 μs in L311, 68.91 to 710.73 μs in L314, 67.93 to 773.42 μs in W311, and 68.08 to 771.40 μs in

W314. These within-environment changes exceed tenfold at identical cardinality; they do not isolate the rejected fraction as the cause.

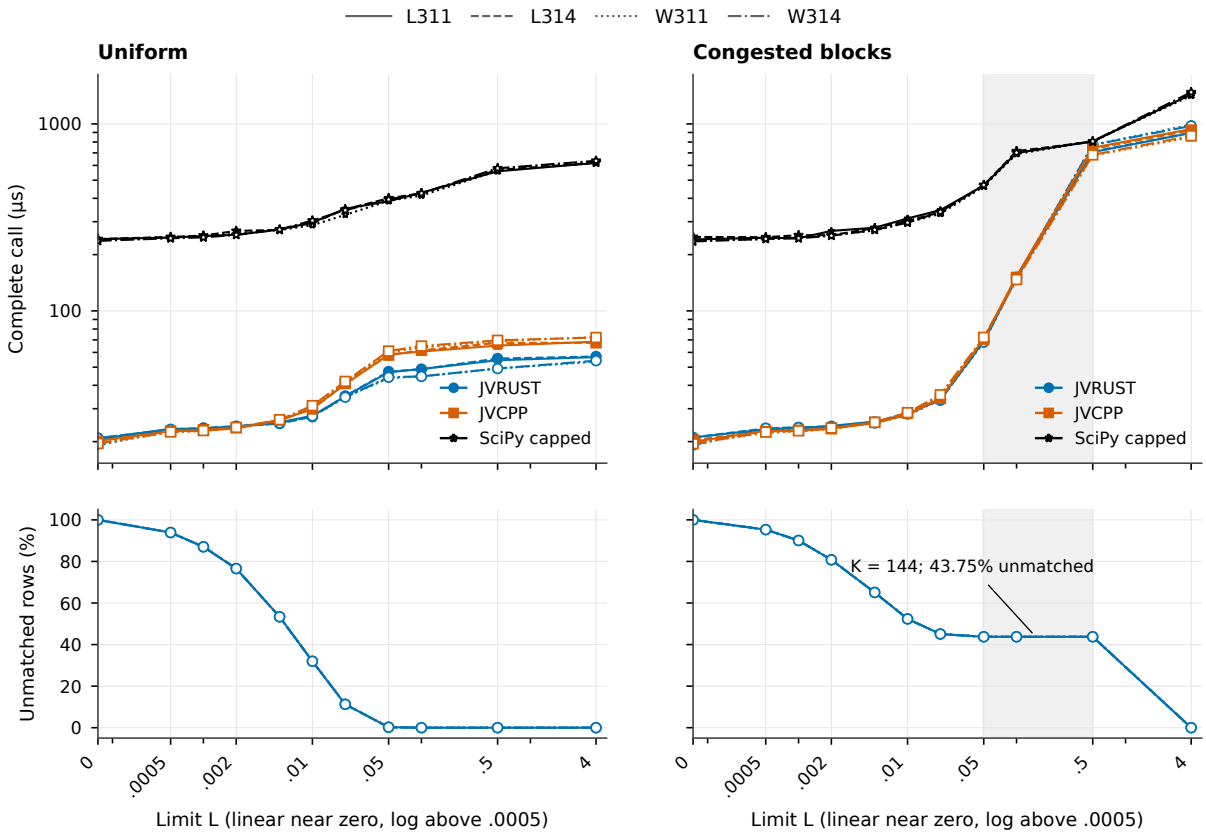


Figure 13: Linked rejection-limit sweeps for uniform and congested-block inputs. Runtime panels retain complete-call latency separately by method and environment; companion panels show the smaller-side unmatched fraction derived from the five saved input cardinalities. Solid/dashed/dotted/dash-dot lines identify L311/L314/W311/W314. The common limit axis is linear from zero through 0.0005 and logarithmic above it. Constant cardinality does not imply constant latency; no solver or input generator was rerun to obtain these values.

C Memory-counter visibility

Each panel retains all 455 fresh-process trials from its environment. Zero high-water increments occur in 420, 420, 88 and 82 trials in L311/L314/W311/W314. In each Linux environment, 80 trials have zero high-water increment despite more than 1 MiB of post-call resident growth; no Windows trial has that combination. This describes visibility under the recorded headroom, not intrinsic solver memory efficiency. The complete memory export retains trial-level counter names, pre/post RSS, peaks, headroom and condition identities.

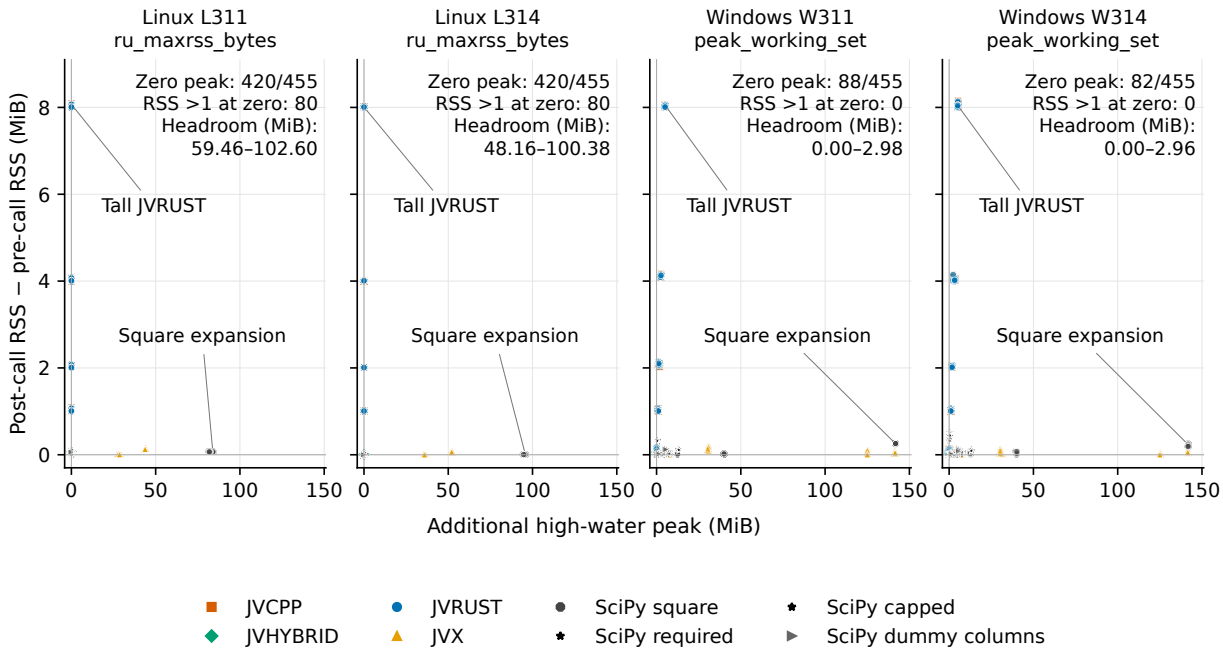


Figure 14: Additional high-water increment versus net post-call resident growth, in MiB, with true zeros retained on linear axes. Each panel identifies its operating-system counter and baseline headroom. Net resident growth is a snapshot difference, not a peak or allocation count.

D Application workload and accuracy

The workload table counts each frozen pipeline once, rather than once per API or environment. Replay uses the common matrices captured through SciPy. Tracker updates can evolve different assignments and predictions; the accuracy table therefore keeps solver identities and differences explicit. All precision views, absolute times, stored speedups, round values, validation details and secondary instrumented solver fractions remain in the application supplement.

Table 5: Frozen application workload inventory. Frames, captured matrices and source identities describe the supplied corpus, not a wider tracking population.

Pipeline	Tracker	Frames	Matrices	Rows	Columns
MOT16-04 / GT	Centroid	999	998	47–56	47–56
MOT16-04 / GT	Sort	999	998	47–56	47–56
MOT16-04 / RF-DETR	Centroid	999	998	22–39	23–39
MOT16-04 / RF-DETR	Sort	999	976	23–39	24–42
MOT20-03 / GT	Centroid	999	998	84–134	84–134
MOT20-03 / GT	Sort	999	998	84–134	84–134
MOT20-03 / RF-DETR	Centroid	999	998	34–62	34–62
MOT20-03 / RF-DETR	Sort	999	575	36–62	39–66

Table 6: Supplied-box tracking validation and metric differences relative to SciPy. Percentage-point changes, switch counts and prediction differences are distinct from replay objective agreement. All retained differences are descriptive and do not establish a general accuracy benefit. Native denotes JVRUST/JVHYBRID/JVCPP; C/S denote Centroid/SORT, M16/M20 the two clips, GT/RF the detection source. The metric vectors and prediction identities agree across environments, so these identical results are printed once.

Pipeline	Solver group	ΔIDF1	ΔMOTA	ΔIDs	ΔFP	ΔFN	Predictions
M16 / gt / C	Native	+0.524	+0.002	-1	+0	+0	Different
M16 / gt / C	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Different
M16 / gt / S	Native	+0.000	+0.000	+0	+0	+0	Same
M16 / gt / S	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Same
M16 / RF / C	Native	+0.000	-0.006	+3	+0	+0	Different
M16 / RF / C	JVX APIs/JVS	+0.333	-0.012	+4	+1	+1	Different
M16 / RF / S	Native	+0.000	+0.000	+0	+0	+0	Same
M16 / RF / S	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Different
M20 / gt / C	Native	+0.000	+0.000	+0	+0	+0	Same
M20 / gt / C	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Same
M20 / gt / S	Native	+0.000	+0.000	+0	+0	+0	Same
M20 / gt / S	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Same
M20 / RF / C	Native	-0.008	-0.001	+1	+0	+0	Different
M20 / RF / C	JVX APIs/JVS	-0.397	-0.002	+2	+0	+0	Different
M20 / RF / S	Native	+0.000	+0.000	+0	+0	+0	Same
M20 / RF / S	JVX APIs/JVS	+0.000	+0.000	+0	+0	+0	Same

E Protocol sensitivity and indexed supplement

Only this protocol-sensitivity analysis uses the separate full Linux/Python 3.11 session. It contains 374 cases, including 340 single matrices and 192 geometry cases, compared with 302, 268 and 120 in each primary light session. The comparison recomputes summaries from saved rounds; it is not an additional benchmark run or a fifth environment in the primary figures.

Taking the first three of five assignment rounds changes none of the 818 native-versus-SciPy practical classifications. Across all ten three-round subsets, at most two classifications change. Truncating application rounds from nine to the first three gives maximum observed speedup shifts that round to 0.71% for tracker updates and 0.38% for replay. These are conditional sensitivity checks on one fixed full session, not equivalence tests. The saved-data supplement also lists the coverage removed by the geometry setting and baseline rules and identifies expensive admitted baselines without extrapolating omitted arms.

The indexed supplement is organized as S1 protocol sensitivity, S2 complete primary conditions and native classifications, S3 policy/layout timing and diagnostic work, S4 applications, S5 batch/workspace and memory, and S6 LSAPE. Records preserve environment, case, method, reference, source file and available attempt/input/build identities. Protocol exclusions retain their reason rather than becoming zero latency or a win. The generator validates coverage and joins before drawing; the four environments remain separate observations throughout.

F Classification tables

The following tables retain the exact descriptive counts behind the distribution plots. They use the same unrounded-ratio classification and keep environments separate. Counts describe case coverage, not effect magnitude or statistical uncertainty.

Table 7: Native optional wins/ties/losses by workload family and environment. Each environment contains 166 optional cases. Required JVRUST/JVHYBRID/JVCPP each win all 102 required cases. Family denominators are retained; environments are not pooled.

	L311		L314		W311		W314	
Family (n)	Rust	C++	Rust	C++	Rust	C++	Rust	C++
Base (5)	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0
Formulations (5)	5/0/0	4/1/0	5/0/0	4/1/0	4/1/0	5/0/0	4/1/0	5/0/0
Scaling (27)	27/0/0	26/1/0	27/0/0	26/1/0	26/1/0	27/0/0	27/0/0	27/0/0
Large square (4)	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0
Large rectangle (6)	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0
Extreme rectangle (6)	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0	6/0/0
Congestion (5)	4/1/0	4/0/1	4/1/0	4/0/1	4/0/1	4/1/0	4/0/1	5/0/0
Forbidden (4)	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0
Ties (4)	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0	4/0/0
Signed (2)	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0
Rejection (22)	22/0/0	22/0/0	22/0/0	22/0/0	22/0/0	22/0/0	22/0/0	22/0/0
Geometry (76)	76/0/0	76/0/0	76/0/0	76/0/0	76/0/0	76/0/0	76/0/0	76/0/0

Table 8: Every workload with at least one optional native practical tie or loss, with reference/method ratios by environment. All three use congested-block costs but belong to distinct formulation, scaling and congestion families. Classification uses unrounded ratios; displayed values are rounded.

Workload	Method	L311	L314	W311	W314
Formulation blocks, 256×320	JVRUST	1.070	1.051	0.971	0.977
Formulation blocks, 256×320	JVCPP	1.017	1.005	1.107	1.096
Scaling blocks, 128×160	JVRUST	1.073	1.101	0.994	1.030
Scaling blocks, 128×160	JVCPP	1.016	1.013	1.103	1.163
Congestion 041, 256×320	JVRUST	0.984	0.985	0.916	0.926
Congestion 041, 256×320	JVCPP	0.956	0.943	1.023	1.036

Table 9: LSAPE wins/ties/losses by family and Windows environment. Counts classify each case's median of three process ratios. O/P is the frozen primary comparison; O/U, U/P, P/T and S/T retain their distinct controlled interpretations.

Family	Env.	n	O/P	O/U	U/P	P/T	S/T
Square	W311	8	0/1/7	0/0/8	8/0/0	0/4/4	0/7/1
Square	W314	8	0/1/7	0/0/8	8/0/0	0/5/3	0/8/0
5:4 uniform	W311	16	4/2/10	4/0/12	16/0/0	0/9/7	0/16/0
5:4 uniform	W314	16	4/3/9	4/0/12	16/0/0	0/9/7	0/16/0
5:4 congested	W311	8	8/0/0	8/0/0	8/0/0	0/5/3	0/8/0
5:4 congested	W314	8	8/0/0	8/0/0	8/0/0	0/5/3	0/8/0
4:1	W311	16	13/3/0	5/1/10	16/0/0	0/10/6	0/16/0
4:1	W314	16	13/3/0	5/2/9	16/0/0	0/8/8	0/16/0
Extreme	W311	12	10/2/0	3/3/6	12/0/0	0/8/4	0/12/0
Extreme	W314	12	10/2/0	3/6/3	12/0/0	0/6/6	0/12/0
Threshold ties	W311	7	3/4/0	0/1/6	7/0/0	0/7/0	0/7/0
Threshold ties	W314	7	3/4/0	0/1/6	7/0/0	0/5/2	0/7/0